ArchitectPoint

Overview:

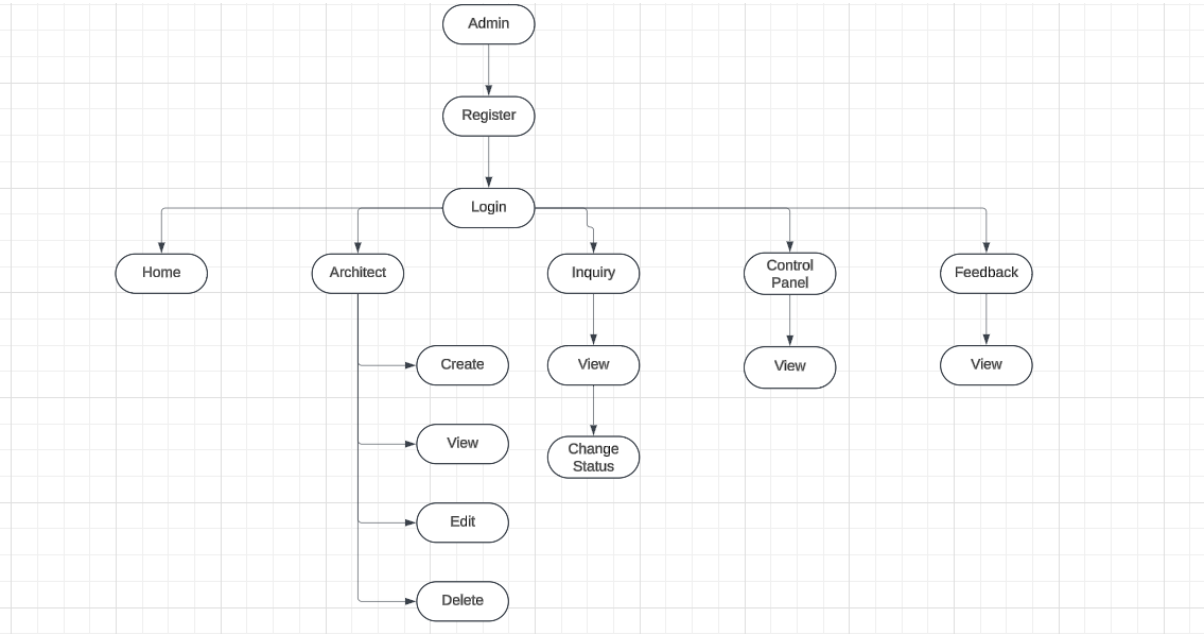
ArchitectPoint is an innovative platform designed to connect users with skilled architects and design services, streamlining the process of managing architect inquiries, portfolio browsing, and collaboration. The application offers a variety of features such as architect listings, user feedback, inquiry handling, and portfolio management, designed to meet diverse user needs. With user-friendly interfaces for both Admins and Users, ArchitectPoint ensures smooth communication and accessibility to facilitate seamless architectural project operations.

Users of the System:

1. Admin
2. User

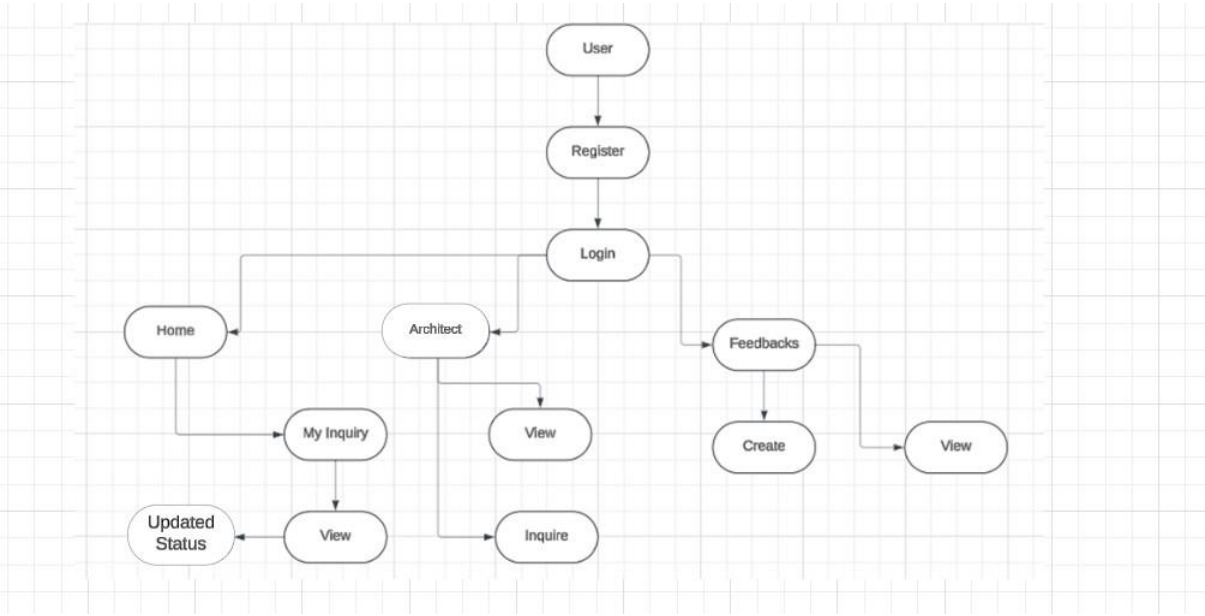
Admin Actions:

Flow Diagram:



User Actions:

Flow Diagram:



Modules of the Application:

Admin:

1. Register
2. Login
3. Home
4. Investment
5. Admin console
6. Inquiry
7. Feedback

User:

1. Register
2. Login
3. Home
4. Investment
5. My Inquiry
6. Feedback

Technology Stack

Front End	Angular 16+, HTML, CSS
Back End	Java, Spring boot, Mysql
Authentication	JWT for User Authentication.

Application assumptions:

1. The login page should be the first page rendered when the application loads.
2. Manual routing should be restricted by implementing Auth Guard, utilizing the canActivate interface. For example, if the user enters as http://localhost:8080/dashboard or

http://localhost:8080/user the page should not navigate to the corresponding page instead it should redirect to the login page.

3. Unless logged into the system, the user cannot navigate to any other pages.
4. Logging out must again redirect to the login page.

Backend Endpoints:

Controller:

AuthController API Endpoints

Action	URL	Method	Role	Response
User Registration	/api/register	POST	All	Return response code 201 with User object on successful creation or else 403 and 409 if failure
User Login	/api/login	POST	All	Return response code 200 with Login object on successful creation or else 403, 409 if failure

ArchitectController API Endpoints

Action	URL	Method	Role	Response
Add **Architect**	/api/architects	POST	Admin	Returns response code 201 with Architect object on successful creation or 403 and 409 if failure
View **Architect** by ID	/api/architects/{architectId}	GET	Admin	Returns response code 200 with Architect object on successful retrieval or

				404 if not found.
View All architects	/api/architects	GET	ALL	Returns response code 200 with a list of Architect objects on successful retrieval or 400 if failure.
Update Architect	/api/architects/{architectId}	PUT	Admin	Returns response code 200 with updated Architect object on successful update or 404 if not found.
Delete Architect	/api/architects/{architectId}	DELETE	Admin	Returns response code 200 on successful deletion and return type void or 404 if not found.

ArchitectInquiryController API Endpoints

Action	URL	Method	Role	Response
Add Architect Inquiry	/api/inquiries	POST	User	Returns response code 201 with Architect Inquiry object on successful creation or 403 and 409 if failure
View Architect	/api/inquiries/{inquiryId}	GET	Admin, User	Returns response code 200 with Architect Inquiry

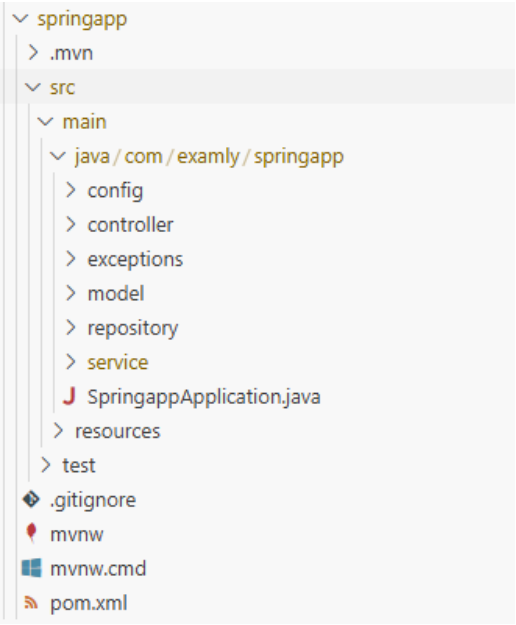
Inquiry by ID				object on successful retrieval or 404 if not found.
View Inquiry by User	/api/inquiries/user/{userId}	GET	User	Returns response code 200 with a list of Architect Inquiry objects on successful retrieval or 404 if failure.
View All Architect Inquiry	/api/inquiries	GET	Admin	Returns response code 200 with a list of Architect Inquiry objects on successful retrieval or 400 if failure.
Update Architect Inquiry	/api/ inquiries/{inquiryId}	PUT	Admin	Returns response code 200 with updated Architect Inquiry object on successful update or 404 if not found.
Delete Architect Inquiry	/api/inquiries/{inquiryId}	DELETE	User	Returns response code 200 on successful deletion and return type void or 404 if not found.

FeedbackController API Endpoints

Action	URL	Method	Role	Response
Create Feedback	/api/feedback	POST	User	Returns response code 201 with Feedback object on successful creation or else 403 and 409 if failure
View Feedback by Id	/api/feedback/{feedbackId}	GET	Admin, User	Returns response code 200 with Feedback object on successful retrieval or else 404 if failure
View All Feedbacks	/api/feedback	GET	Admin	Returns response code 200 with List<Feedback> object on successful retrieval or else 400 if failure
View Feedbacks by User	/api/feedback/user/{userId}	GET	User	Returns response code 200 with List<Feedback> object on successful retrieval or else 404 if failure
Delete Feedback	/api/feedback/{feedbackId}	DELETE	User	Returns response code 200 on successful deletion and return type void or 404 if not found.

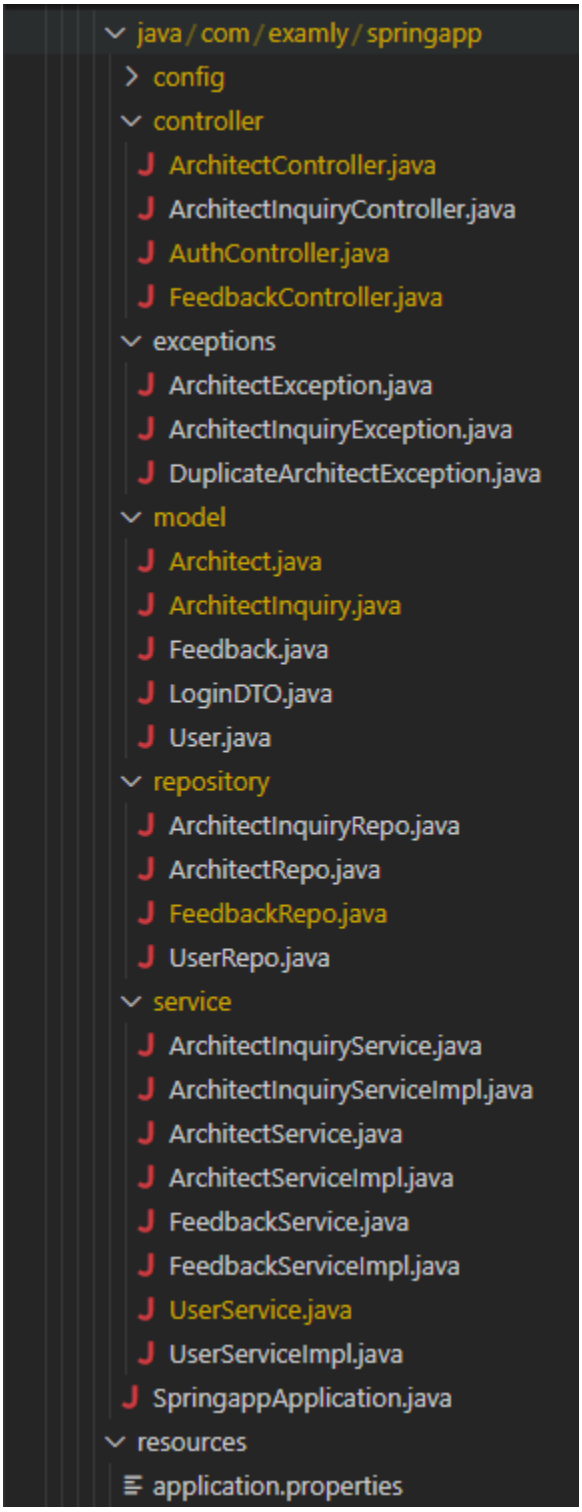
Backend Folder Structure:

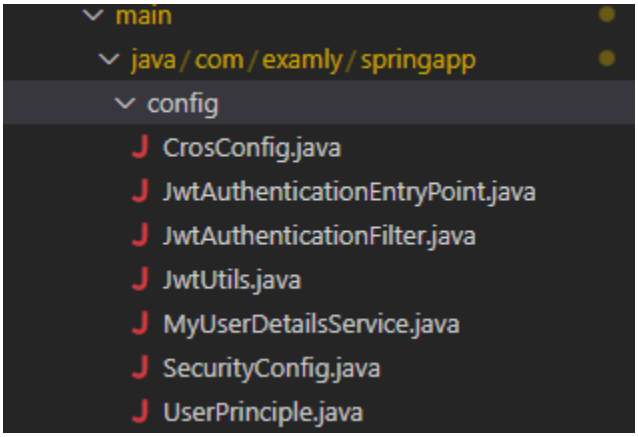
Create folders named as model, repository, service, controller and config inside springapp as mentioned in the below screenshot.



In config folder, create a class CrosConfig with @Configuration annotation which implements WebMvcConfigurer to enable CORS functionality and also add all files related to JWT.

- In controller folder, create classes named **AuthController, ArchitectController, ArchitectInquiryController, FeedbackController** implement REST APIs (Refer the API table).
- In repository folder, create the interfaces named as **UserRepo, FeedbackRepo, ArchitectRepo, ArchitectInquiryRepo** which extends **JpaRepository**.
- Ensure adherence to the specified API outlined in the API table, ensuring consistency in class names, variables, and folder structure as detailed.


java / com / examly / springapp
config
controller
ArchitectController.java
ArchitectInquiryController.java
AuthController.java
FeedbackController.java
exceptions
ArchitectException.java
ArchitectInquiryException.java
DuplicateArchitectException.java
model
Architect.java
ArchitectInquiry.java
Feedback.java
LoginDTO.java
User.java
repository
ArchitectInquiryRepo.java
ArchitectRepo.java
FeedbackRepo.java
UserRepo.java
service
ArchitectInquiryService.java
ArchitectInquiryServiceImpl.java
ArchitectService.java
ArchitectServiceImpl.java
FeedbackService.java
FeedbackServiceImpl.java
UserService.java
UserServiceImpl.java
SpringappApplication.java
resources
application.properties



Model Classes

Create a model folder inside the springapp. Inside the model folder, create all the classes mentioned below.

User Class

This class stores all user information.

- **Fields:**
  - userId: Long (auto-generated, primary key)
  - email: String
  - password: String
  - username: String
  - mobileNumber: String
  - userRole: String ("Admin" or "User")

Architect Class

This class stores all architect information.

- **Fields:**
  - architectId: Long (auto-generated, primary key)
  - name: String
  - expertise: String (e.g., "Residential", "Commercial")
  - location: String (Architect's working location or office address)
  - rating: Double (Average rating of the architect)
  - portfolioLink: String (Link to architect's portfolio or website)
  - status: String (e.g., "Active", "Inactive")

ArchitectInquiry Class

This class stores all architect inquiry information.

- **Fields:**
  - inquiryId: Long (auto-generated, primary key)
  - user: User (@ManyToOne, @JoinColumn(name = "userId", nullable = false))
  - architect: Architect (@ManyToOne, @JoinColumn(name = "architectId", nullable = false))
  - message: String (User's inquiry message)
  - status: String ("Pending", "Resolved", "Closed")
  - priority: String ("Low", "Medium", "High")
  - contactDetails: String (User's contact information)

Feedback Class

This class stores all feedback information.

- **Fields:**
  - feedbackId: Long (auto-generated, primary key)
  - feedbackText: String
  - date: String (Feedback submission date in ISO format)
  - user: User (@ManyToOne, @JoinColumn(name = "userId", nullable = false))
  - architect: Architect (@ManyToOne, @JoinColumn(name = "architectId", nullable = true))
  - category: String (e.g., "Service", "Pricing", "Quality")

DTO Classes

LoginDTO Class:

- **Fields:**
  - token: String (JWT token)
  - username: String (Username of the logged-in user)
  - userRole: String (Role of the user, e.g., "Admin" or "User")
  - userId: Long (User's unique ID)

Interfaces to be used:

UserService Interface

- **Methods:**
  1. **createUser(User user):**
     - Description: Creates a new user in the ArchitectPoint system.
     - Parameters: User user - The user object to be created.
     - Return Type: void
  2. **loginUser(User user):**
     - Description: Logs in a user into ArchitectPoint.
     - Parameters: User user - The user object containing login credentials.
     - Return Type: Optional<User> - Returns a user object if credentials match.

ArchitectService Interface

- **Methods:**
  1. **addArchitect(Architect architect):**
     - Description: Adds a new architect to ArchitectPoint.
     - Parameters: Architect architect - The architect object to be added.
     - Return Type: Architect
  2. **updateArchitect(Long architectId, Architect updatedArchitect):**
     - Description: Updates an architect's information.
     - Parameters:
       - Long architectId - The ID of the architect to update.
       - Architect updatedArchitect - The updated architect object.
     - Return Type: Architect
  3. **getArchitectById(Long architectId):**
     - Description: Retrieves an architect by their ID.
     - Parameters: Long architectId - The ID of the architect.
     - Return Type: Optional<Architect>
  4. **getAllArchitects():**
     - Description: Retrieves a list of all architects.
     - Parameters: None.
     - Return Type: List<Architect>

5. **getArchitectsByExpertise(String expertise):**
    - Description: Retrieves architects by their expertise.
    - Parameters: String expertise - The expertise to search architects by.
    - Return Type: List<Architect>
6. **getArchitectsByStatus(String status):**
    - Description: Retrieves architects by their status (e.g., Active or Inactive).
    - Parameters: String status
    - Return Type: List<Architect>
7. **searchArchitects(String keyword):**
    - Description: Searches architects by keywords (e.g., name, expertise).
    - Parameters: String keyword
    - Return Type: List<Architect>
8. **getArchitectsByLocation(String location):**
    - Description: Retrieves architects by their location.
    - Parameters: String location
    - Return Type: List<Architect>
9. **deleteArchitect(Long architectId):**
    - Description: Deletes an architect by their ID.
    - Parameters: Long architectId - The ID of the architect to delete.
    - Return Type: void

ArchitectInquiryService Interface

- **Methods:**
    1. **createInquiry(ArchitectInquiry architectInquiry):**
        - Description: Adds a new architect inquiry.
        - Parameters: ArchitectInquiry architectInquiry - The inquiry object to be added.
        - Return Type: ArchitectInquiry
    2. **updateInquiry(Long inquiryId, ArchitectInquiry updatedInquiry):**
        - Description: Updates an architect inquiry.
        - Parameters:
            - Long inquiryId - The ID of the inquiry to update.
            - ArchitectInquiry updatedInquiry - The updated inquiry object.
        - Return Type: ArchitectInquiry
    3. **getInquiriesByUserId(Long userId):**
        - Description: Retrieves inquiries made by a specific user.
        - Parameters: Long userId
        - Return Type: List<ArchitectInquiry>
    4. **getInquiriesByArchitectId(Long architectId):**
        - Description: Retrieves inquiries made for a specific architect.
        - Parameters: Long architectId
        - Return Type: List<ArchitectInquiry>
    5. **getUnresolvedInquiries():**
        - Description: Retrieves all unresolved inquiries.
        - Parameters: None.
        - Return Type: List<ArchitectInquiry>
    6. **getInquiriesByPriority(String priority):**
        - Description: Retrieves inquiries filtered by priority.
        - Parameters: String priority
        - Return Type: List<ArchitectInquiry>
    7. **getAllInquiries():**

- Description: Retrieves all inquiries.
- Parameters: None.
- Return Type: List<ArchitectInquiry>

8. **deleteInquiry(Long inquiryId):**
    - Description: Deletes an inquiry by its ID.
    - Parameters: Long inquiryId
    - Return Type: void

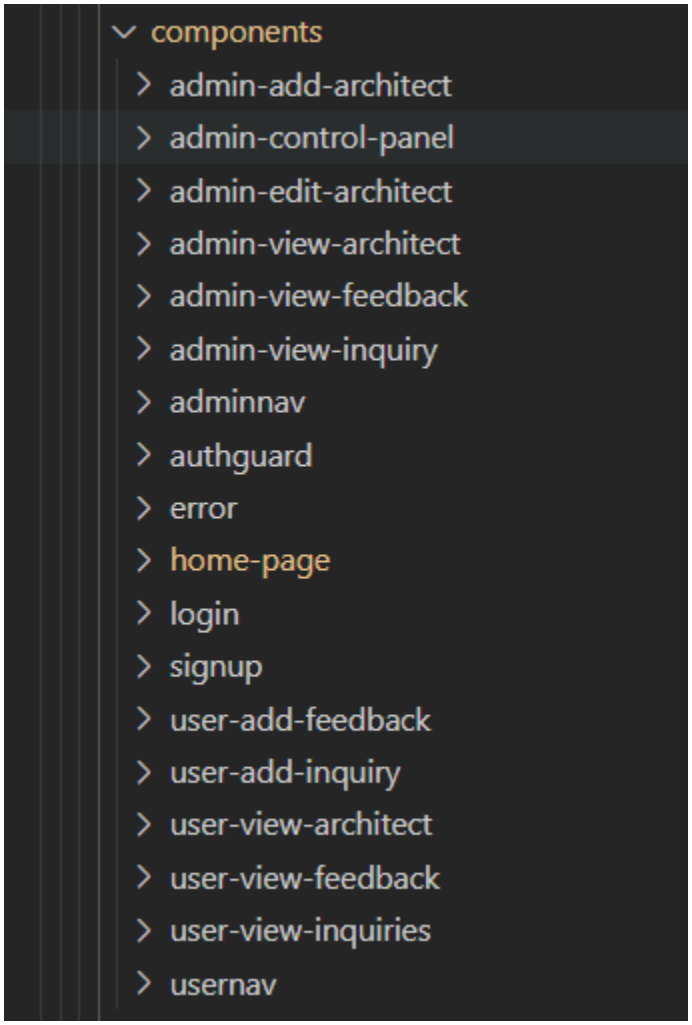
FeedbackService Interface

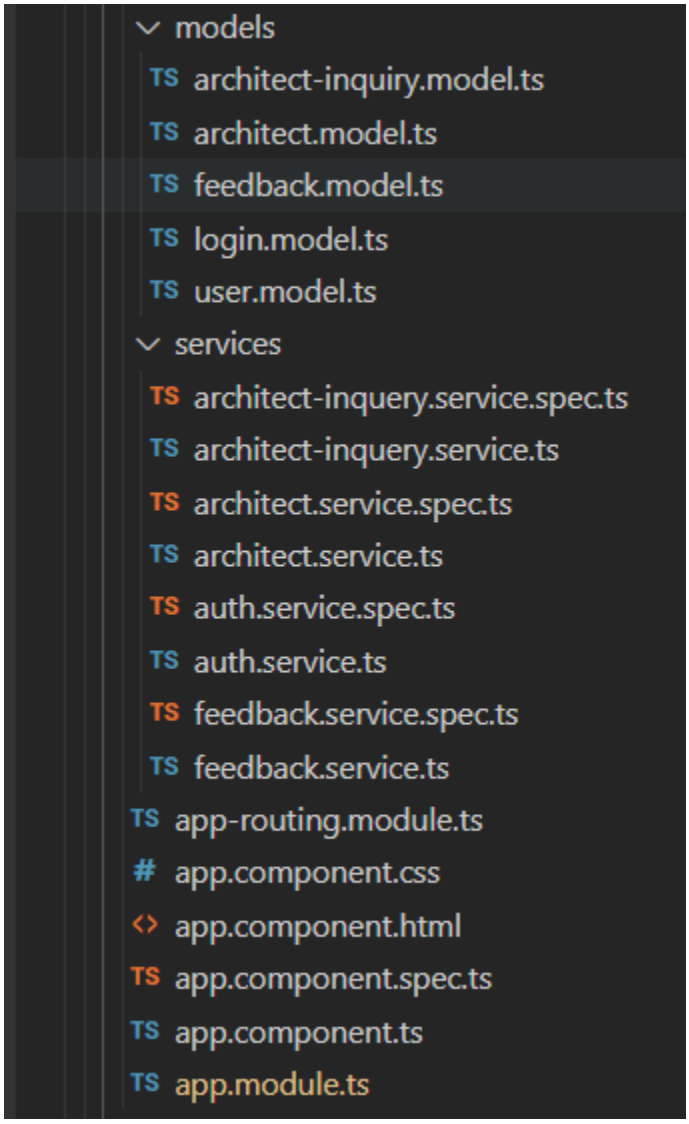
- **Methods:**
    1. **createFeedback(Feedback feedback):**
        - Description: Creates a new feedback.
        - Parameters: Feedback feedback - The feedback object to be created.
        - Return Type: Feedback
    2. **getFeedbackById(Long feedbackId):**
        - Description: Retrieves feedback details by its ID.
        - Parameters: Long feedbackId
        - Return Type: Optional<Feedback>
    3. **getAllFeedbacks():**
        - Description: Retrieves a list of all feedback.
        - Parameters: None.
        - Return Type: List<Feedback>
    4. **deleteFeedback(Long feedbackId):**
        - Description: Deletes feedback by its ID.
        - Parameters: Long feedbackId
        - Return Type: void
    5. **getFeedbacksByUserId(Long userId):**
        - Description: Retrieves feedback submitted by a specific user.
        - Parameters: Long userId
        - Return Type: List<Feedback>
    6. **getFeedbacksByArchitectId(Long architectId):**
        - Description: Retrieves feedback submitted for a specific architect.
        - Parameters: Long architectId
        - Return Type: List<Feedback>

Frontend Requirements:

- Create a folder named components inside the app to store all the components. (Refer project structure screenshots).
- Create a folder named models inside app to store all the model interface.
- Create a folder named as services inside app to implement all the services.
- Create model interface referring the backend entities mentioned in the backend requirements accordingly.
- You can create your own components based on the application requirements.
- Import model files, services and components as required.

Project Folder Screenshot:


components
admin-add-architect
admin-control-panel
admin-edit-architect
admin-view-architect
admin-view-feedback
admin-view-inquiry
adminnav
authguard
error
home-page
login
signup
user-add-feedback
user-add-inquiry
user-view-architect
user-view-feedback
user-view-inquiries
usernav



Frontend Models: User Model:

```
class User {
  userId?: number;
  email: string;
  password: string;
  username: string;
  mobileNumber: string;
  userRole: string;
}
```

Login Model:

```
class Login {
email: string;
password: string;
```

```
}
```

Architect Model:

```
interface Architect {
  architectId?: number; // Unique identifier for the architect
  name: string; // Name of the architect
  expertise: string; // Architect's expertise (e.g., "Residential", "Commercial")
  location: string; // Architect's location or office address
  rating: number; // Average rating of the architect
  portfolioLink: string; // Link to the architect's portfolio or website
  status: string; // Status (e.g., "Active", "Inactive")
}
```

ArchitectInquiry Model:

```
interface ArchitectInquiry {
  inquiryId?: number; // Make optional for new inquiries
  user: Partial<User>; // Allow partial User object
  architect: Partial<Architect>; // Allow partial Architect object
  message: string;
  priority: string;
  contactDetails?: string; // Optional for new inquiries
}
```

Feedback Model:

```
interface Feedback {
  feedbackId?: number; // Optional because it's auto-generated
  feedbackText: string; // The text content of the feedback
  date: string; // Feedback date in string format (ISO 8601)
  user: { userId: number }; // User object containing userId
  architect?: { architectId: number }; // Architect object (optional, depending on the feedback
context)
  category: string; // Feedback category (e.g., "Service", "Pricing")
}
```

Frontend services:

- Declare a public property apiUrl to store the backend URL in all the services.
- For example, public apiUrl = 'http://localhost:8080'. Instead of 'localhost', replace it with the URL of your workspace port 8080 URL.
- For the API's to be used please refer the API Table.
- Authorized token to be passed in headers for all end points.

1. AuthService (auth.service.ts)
File Path: app/services/auth.service.ts

Methods Overview

1. **register(user: User): Observable<any>**
    - Description: Registers a new user.
    - Request:
        - URL: /api/register
        - Method: POST
        - Body: User object containing user details.
    - Response: Returns a response object on success; returns error codes 403 or 409 on failure.
2. **login(login: Login): Observable<any>**
    - Description: Authenticates and logs in a user.
    - Request:
        - URL: /api/login
        - Method: POST
        - Body: Login object containing email and password.
    - Post-Login Operations:
        - Stores JWT token in localStorage.
        - Updates user role and ID using BehaviorSubjects.

ArchitectInquiryService Interface
File Path: app/services/architect-inquiry.service.ts
Methods Overview:

1. **addInquiry(inquiry: ArchitectInquiry):**
    - **Description: Adds a new inquiry related to an architect.**
    - **Request:**
        - **URL: /api/inquiries**
        - **Method: POST**
        - **Body: ArchitectInquiry object**
        - **Headers: Authorization token prefixed with Bearer.**
2. **getAllInquiries():**
    - **Description: Fetches all inquiries (Admin only).**
    - **Request:**
        - **URL: /api/inquiries**
        - **Method: GET**
        - **Headers: Authorization token prefixed with Bearer.**
3. **getInquiriesByUserId(userId: number):**
    - **Description: Retrieves inquiries made by a specific user.**
    - **Request:**
        - **URL: /api/inquiries/user/:userId**
        - **Method: GET**
        - **Parameters: userId**
        - **Headers: Authorization token prefixed with Bearer.**
4. **updateInquiry(inquiryId: number, inquiry: ArchitectInquiry):**
    - **Description: Updates an inquiry (Admin only).**
    - **Request:**
        - **URL: /api/inquiries/:inquiryId**
        - **Method: PUT**
        - **Body: Updated ArchitectInquiry object**
        - **Headers: Authorization token prefixed with Bearer.**
5. **deleteInquiry(inquiryId: number):**
    - **Description: Deletes an inquiry by its ID (Admin only).**

- **Request:**
  - **URL: /api/inquiries/:inquiryId**
  - **Method: DELETE**
  - **Parameters: inquiryId**
  - **Headers: Authorization token prefixed with Bearer.**

FeedbackService Interface
File Path: app/services/feedback.service.ts
Methods Overview:

1. **sendFeedback(feedback: Feedback):**
   - **Description: Sends feedback related to an architect or service.**
   - **Request:**
     - **URL: /api/feedback**
     - **Method: POST**
     - **Body: Feedback object**
     - **Headers: Authorization token prefixed with Bearer.**
2. **getAllFeedbacksByUserId(userId: number):**
   - **Description: Retrieves feedback submitted by a specific user.**
   - **Request:**
     - **URL: /api/feedback/user/:userId**
     - **Method: GET**
     - **Parameters: userId**
     - **Headers: Authorization token prefixed with Bearer.**
3. **deleteFeedback(feedbackId: number):**
   - **Description: Deletes a feedback entry.**
   - **Request:**
     - **URL: /api/feedback/:feedbackId**
     - **Method: DELETE**
     - **Parameters: feedbackId**
     - **Headers: Authorization token prefixed with Bearer.**
4. **getFeedbacks():**
   - **Description: Retrieves all feedback entries.**
   - **Request:**
     - **URL: /api/feedback**
     - **Method: GET**
     - **Headers: Authorization token prefixed with Bearer.**

ArchitectService Interface
File Path: app/services/architect.service.ts
Methods Overview:

1. **getAllArchitects():**
   - **Description: Retrieves all architects.**
   - **Request:**
     - **URL: /api/architects**
     - **Method: GET**
     - **Headers: Authorization token prefixed with Bearer.**
2. **getArchitectById(architectId: number):**
   - **Description: Fetches an architect by ID.**
   - **Request:**
     - **URL: /api/architects/:architectId**
     - **Method: GET**

        - **Parameters: architectId**
        - **Headers: Authorization token prefixed with Bearer.**
3. **addArchitect(architect: Architect):**
    - **Description: Adds a new architect to the system.**
    - **Request:**
        - **URL: /api/architects**
        - **Method: POST**
        - **Body: Architect object**
        - **Headers: Authorization token prefixed with Bearer.**
4. **updateArchitect(architectId: number, architect: Architect):**
    - **Description: Updates an architect's details.**
    - **Request:**
        - **URL: /api/architects/:architectId**
        - **Method: PUT**
        - **Body: Updated Architect object**
        - **Headers: Authorization token prefixed with Bearer.**
5. **deleteArchitect(architectId: number):**
    - **Description: Deletes an architect from the system.**
    - **Request:**
        - **URL: /api/architects/:architectId**
        - **Method: DELETE**
        - **Parameters: architectId**
        - **Headers: Authorization token prefixed with Bearer.**

AuthService Interface
File Path: app/services/auth.service.ts
Methods Overview:

1. **register(user: User):**
    - **Description: Registers a new user.**
    - **Request:**
        - **URL: /api/register**
        - **Method: POST**
        - **Body: User object**
        - **Headers: None.**
2. **login(login: Login):**
    - **Description: Logs a user into the application.**
    - **Request:**
        - **URL: /api/login**
        - **Method: POST**
        - **Body: Login object (contains credentials).**
        - **Headers: None.**
        -

Validations:
Client-Side Validation:

- Implement client-side validation using HTML5 attributes and JavaScript to validate user input before making API requests.
- Provide immediate feedback to users for invalid input, such as displaying error messages near the input fields.

Server-Side Validation:

- Implement server-side validation in the controllers to ensure data integrity.
- Validate user input and API responses to prevent unexpected or malicious data from affecting the application.
- Return appropriate validation error messages to the user interface for any validation failures.

Exception Handling:

- Implement exception handling mechanisms in the controllers to gracefully handle errors and exceptions. Define custom exception classes for different error scenarios, such as API communication errors or database errors.
- Log exceptions for debugging purposes while presenting user-friendly error messages to users. Record all the exceptions and errors handled store in separate table “ErrorLogs”.

Error Pages:

Create custom error pages for different HTTP status codes (e.g., 404 Not Found, 500 Internal Server Error) to provide a consistent and user-friendly error experience. Ensure that error pages contain helpful information and guidance for users.

Thus, create a reliable and user-friendly web application that not only meets user expectations but also provides a robust and secure experience, even when faced with unexpected situations. Error page has to be displayed if something goes wrong.

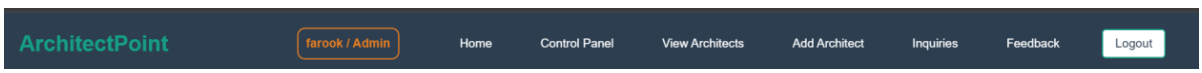


Frontend Screenshots:

• All the asterisk (*) marked fields are mandatory in the form. Make sure to mark all the field names with * symbol followed by the validations.

NavBar:

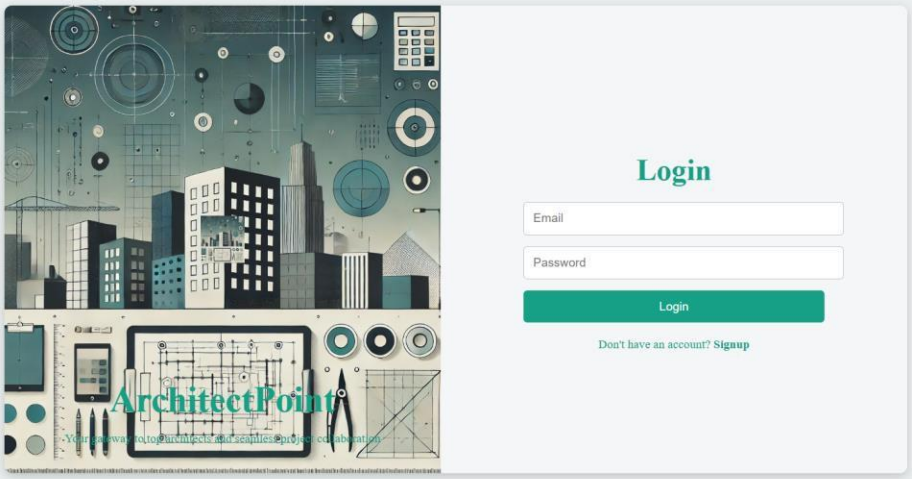
(**adminnav component**)



(**usernav component**)



Landing Page:
(**login component**)

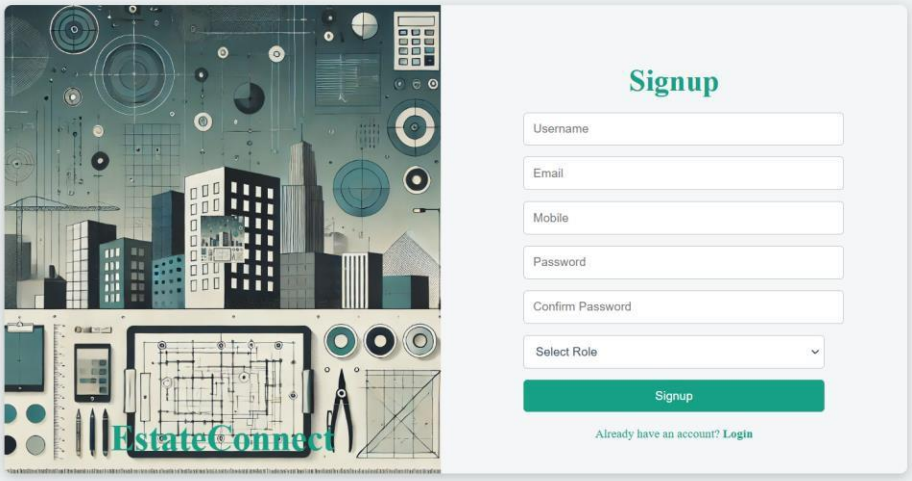


Component features a heading “ArchitectPoint" with background image (background.webp provided in image folder under assets)

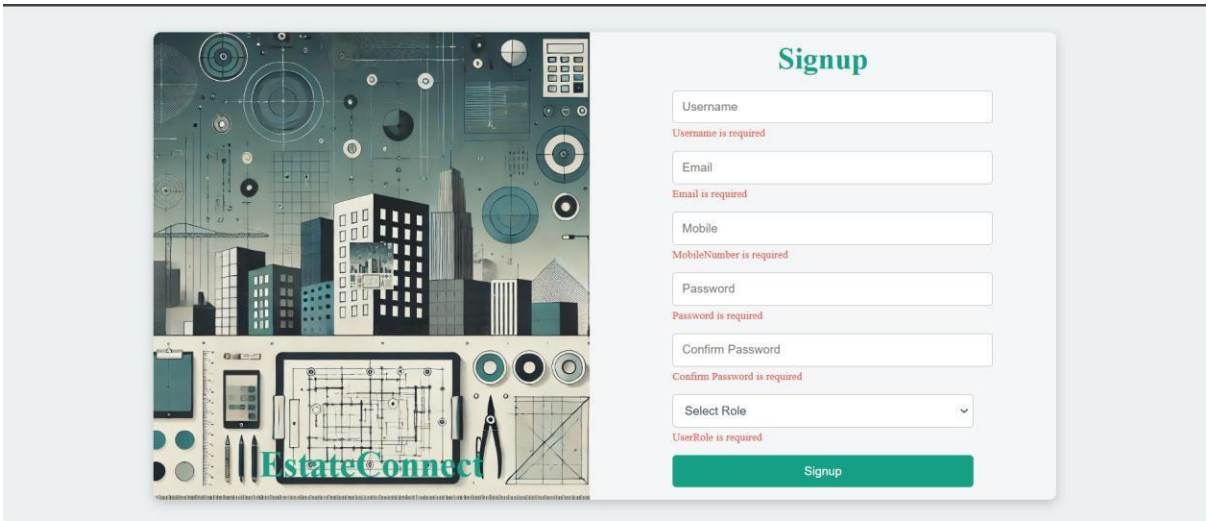
Common Screens (Admin and User)

Signup Page:
(**signup component**)

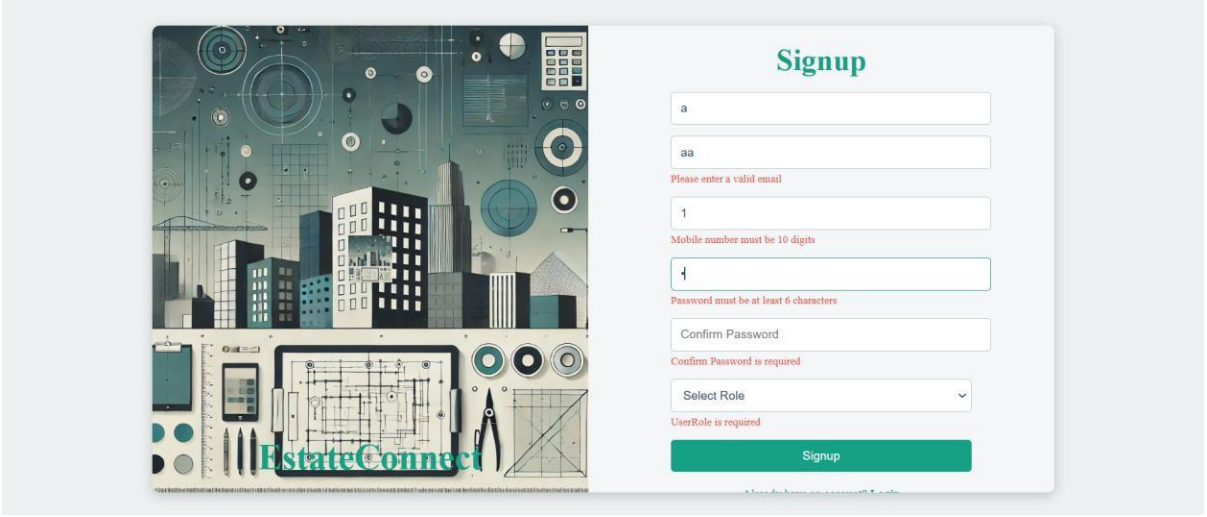
Clicking "**Signup**" in the login page will navigate to the registration page for both roles. Please refer to the screenshots below for validations.



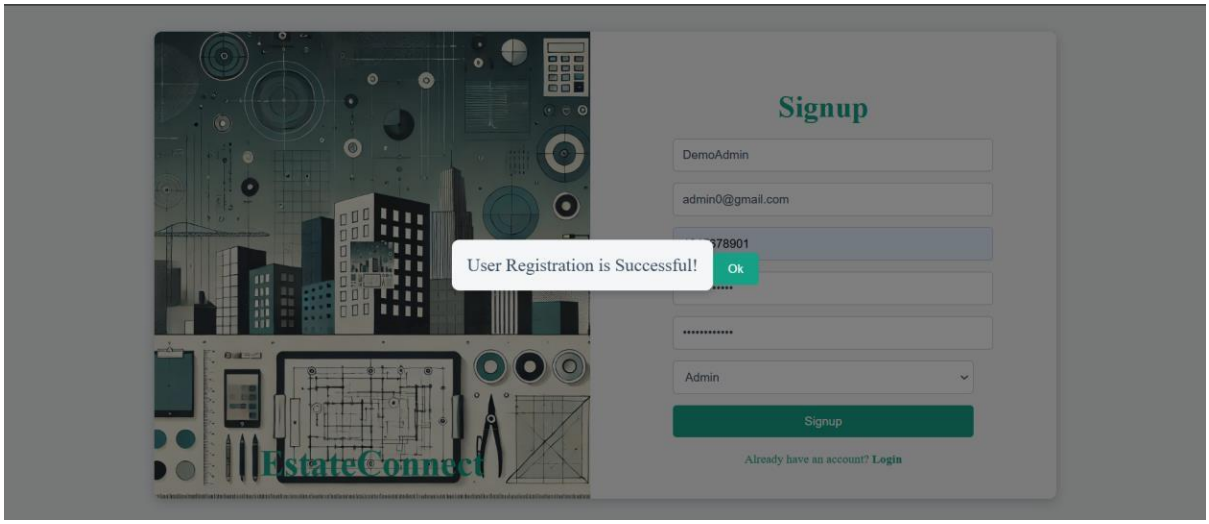
On clicking “Signup” button with empty field will display the validation message.



Regular expressions (Regex) are also included to validate email formats, mobile numbers, and to ensure password matching.

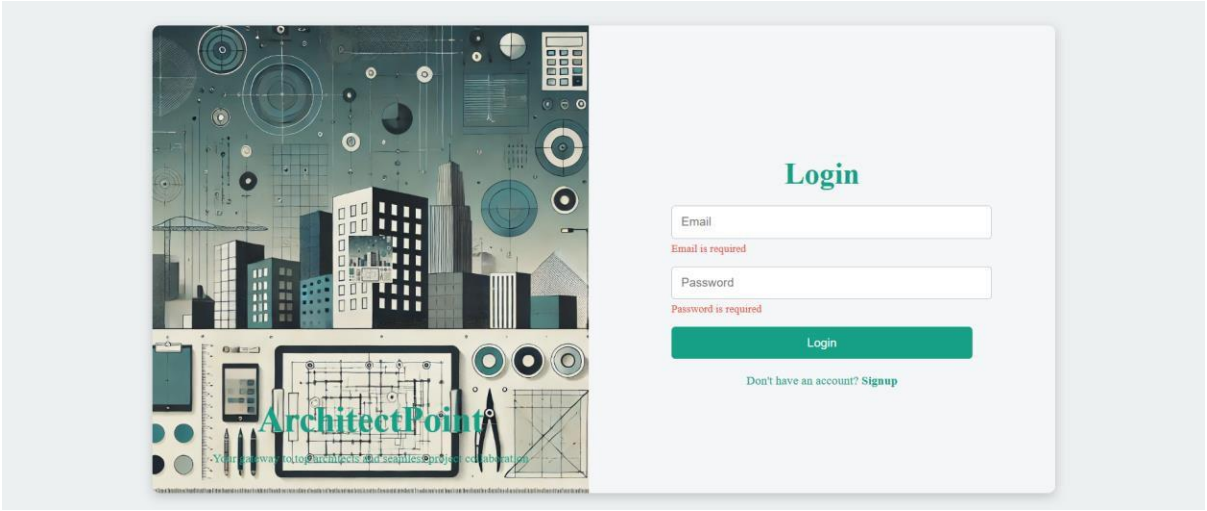


When the "Signup" button is clicked for valid data, upon successful completion, a modal will appear, prompting users to click the "Ok" button. Upon clicking this button, users will be directed to the login page, signifying a successful signup process.



Login Page
(**login component**)

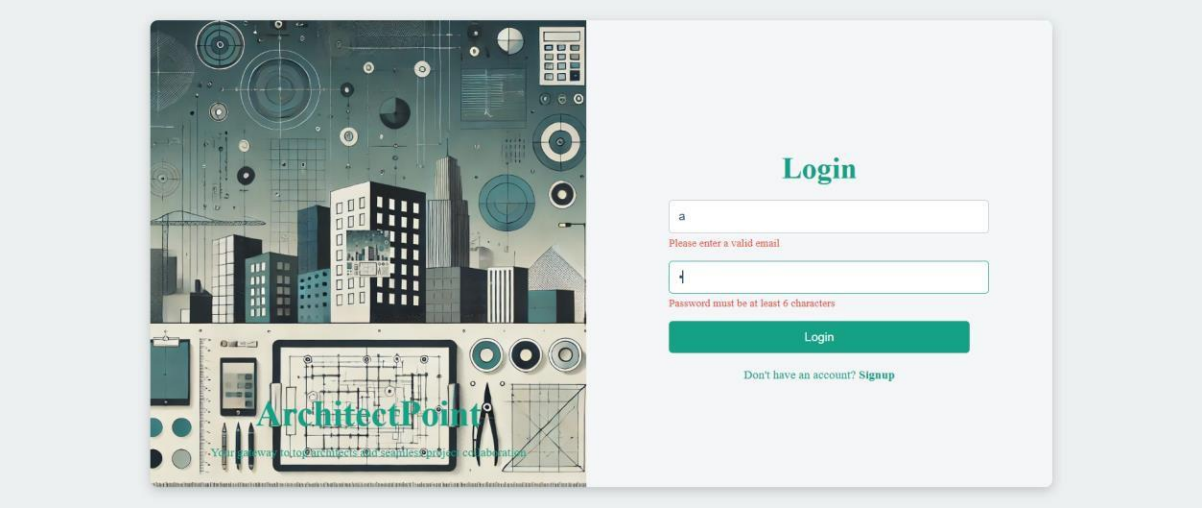
This page is used for logging in to the application. On providing the valid email and password, the user will be logged in.



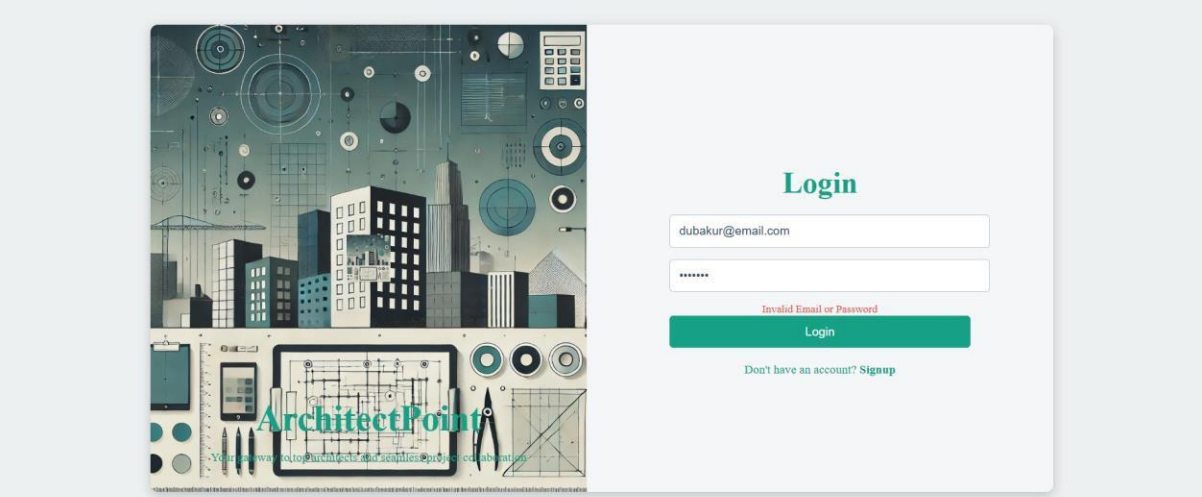
Perform validations for email and password fields.

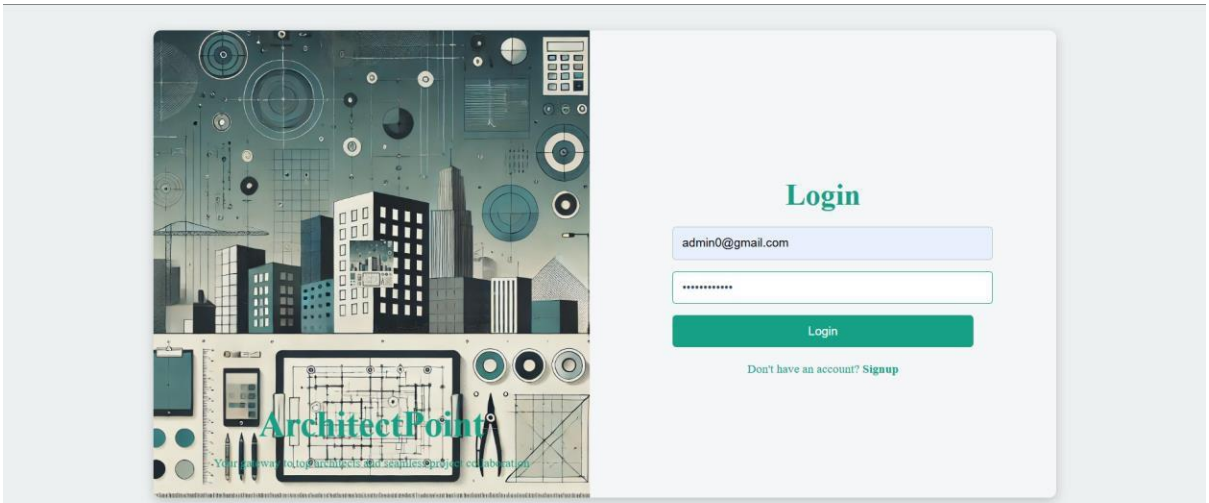
On clicking “Login” button with empty field will display the validation message.

Regex validation:



Display the error message for Invalid Email or Password:



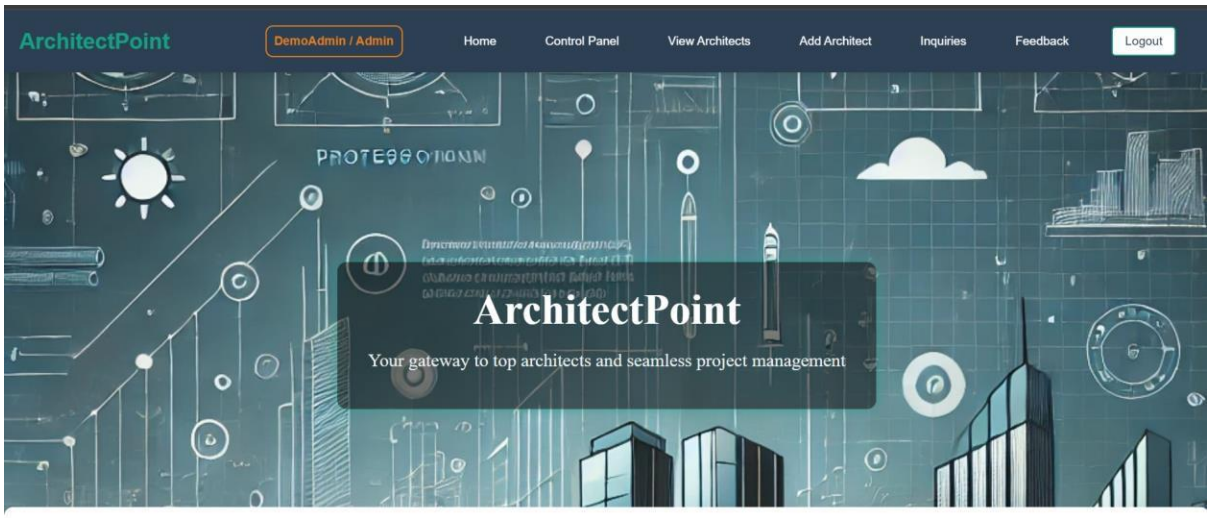


On Clicking the “Login” button, user will be navigated to the (adminnav or usernav component) based on their roles.

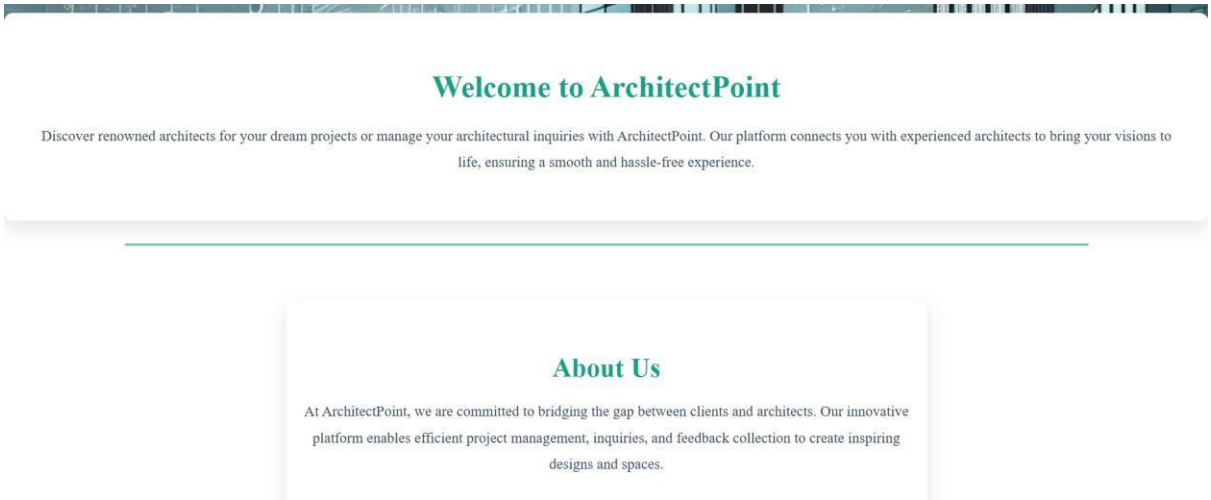
Admin side:

(adminnav component)

Upon successful login, if the user is an User, the (usernav component) will be displayed. If the user is Admin, the (adminnav component) will be displayed. Additionally, the role-based navigation bar will also display login information such as the username and role.

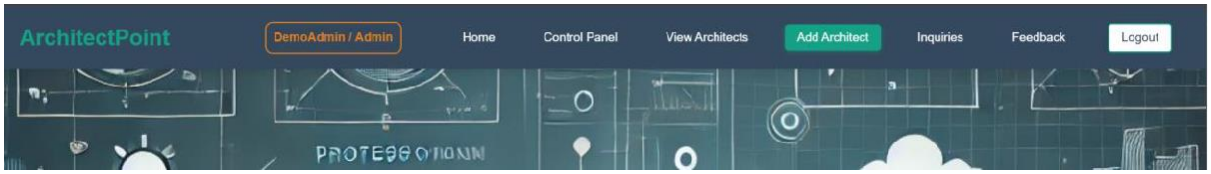


homePage Component: This page is used to display the information about ArchitectPoint application. On clicking the '**Home**' tab, user can view the information about the application.(home.webp provided in images folder under assets)

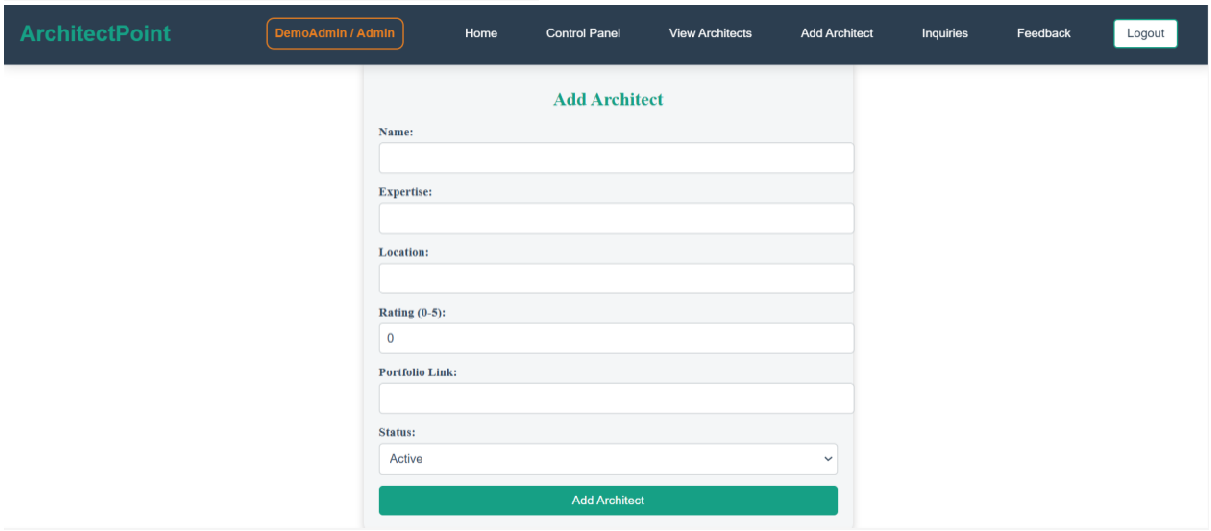


Admins can navigate to other pages by clicking on the menu available in the navigation bar.

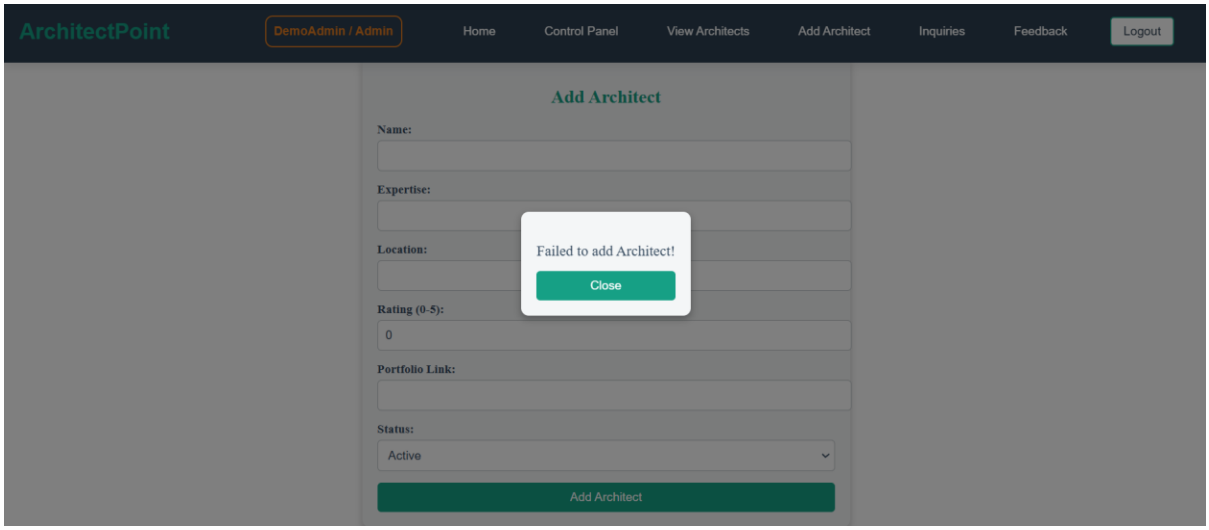
On nav bar, a menu should appear with options to "Add Architect".



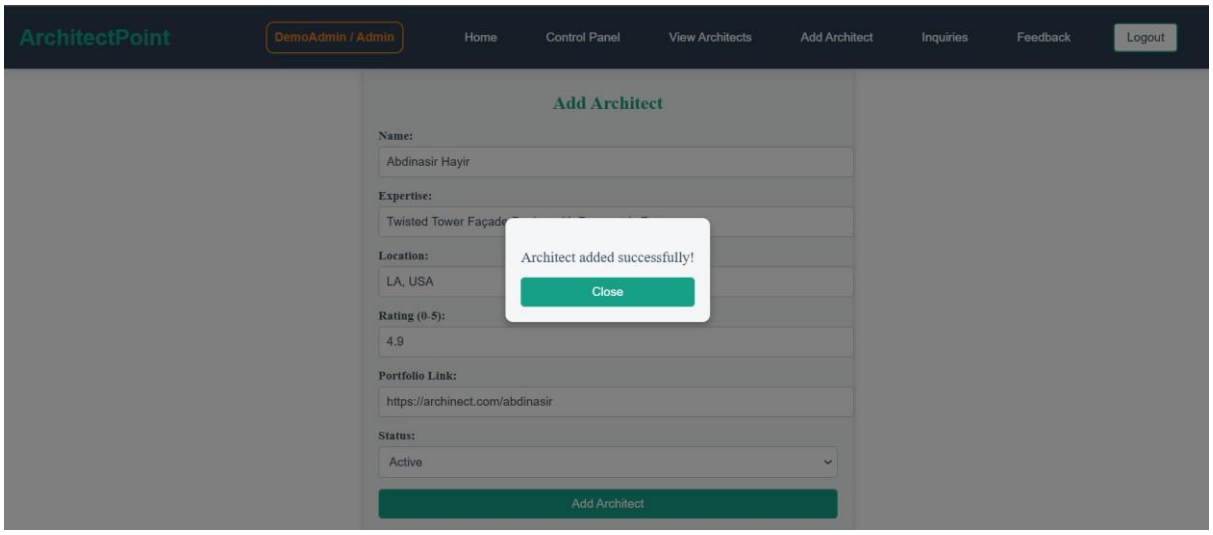
Clicking on "Add Architect" will navigate to the admin-add-architect component, which displays a form with the heading "Add New Architect ".



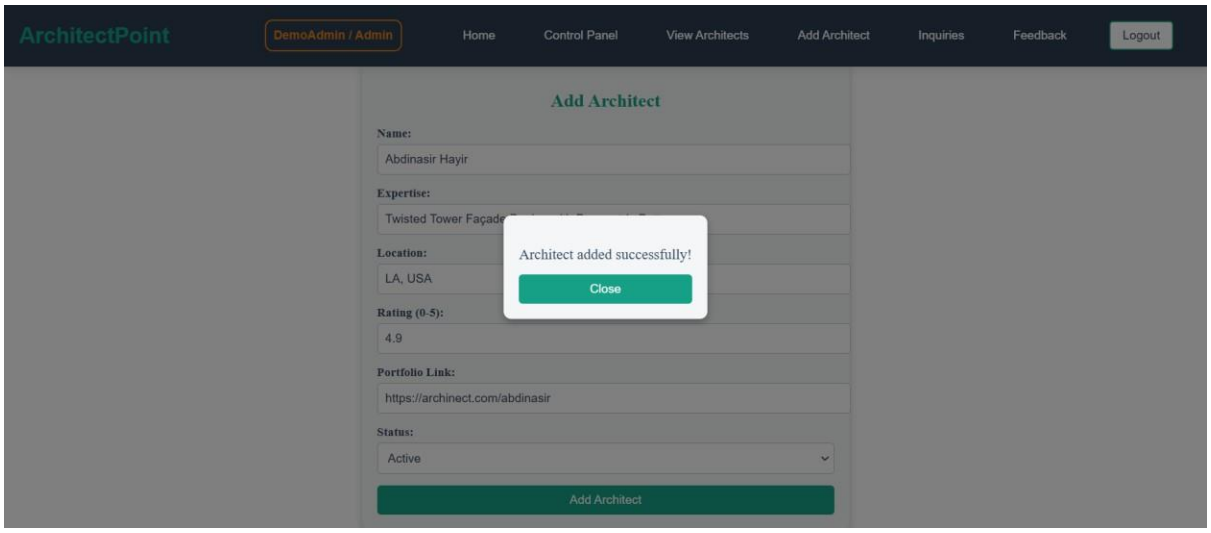
Perform validations for all the form fields.



Clicking the "Add Architect" button with empty fields will display a message stating "Failed to add Architect!".

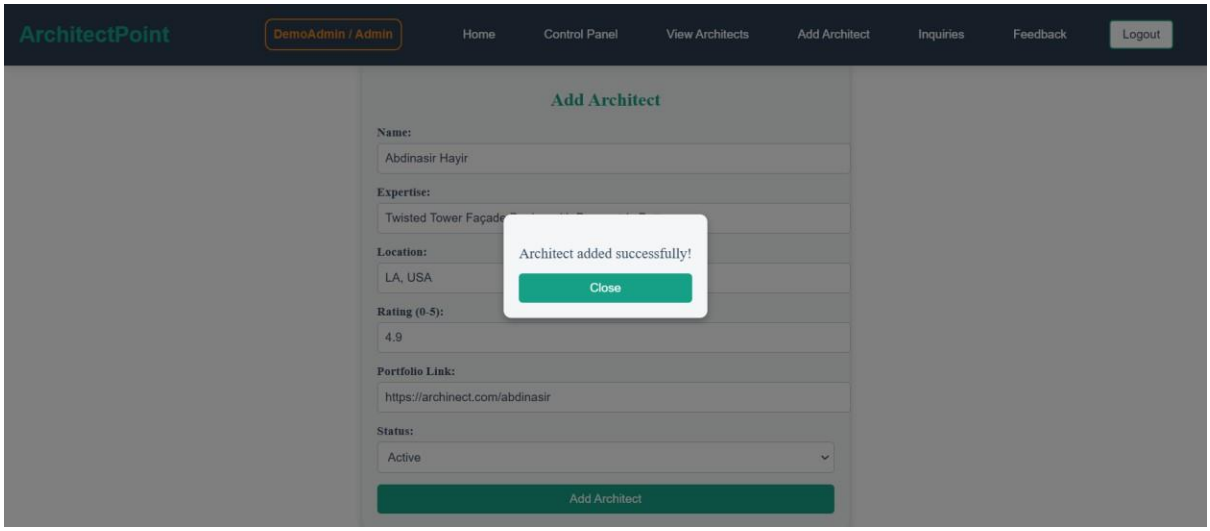


Upon clicking the "Add Architect" button, if the operation is successful, a popup message saying, "Architect Added Successfully!" should be displayed.



On Clicking the “Close” button a architect is added to the system and admin can add a new architect again. To move to other pages admin can click any of the menus available in the navbar.

Admin View



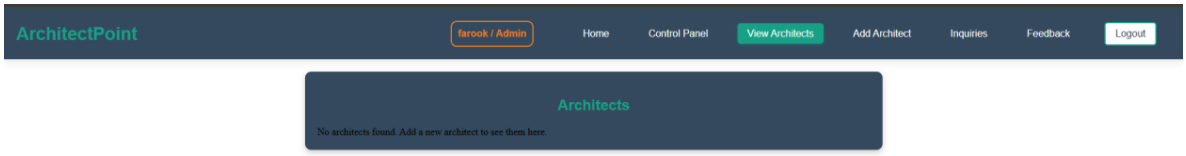
:

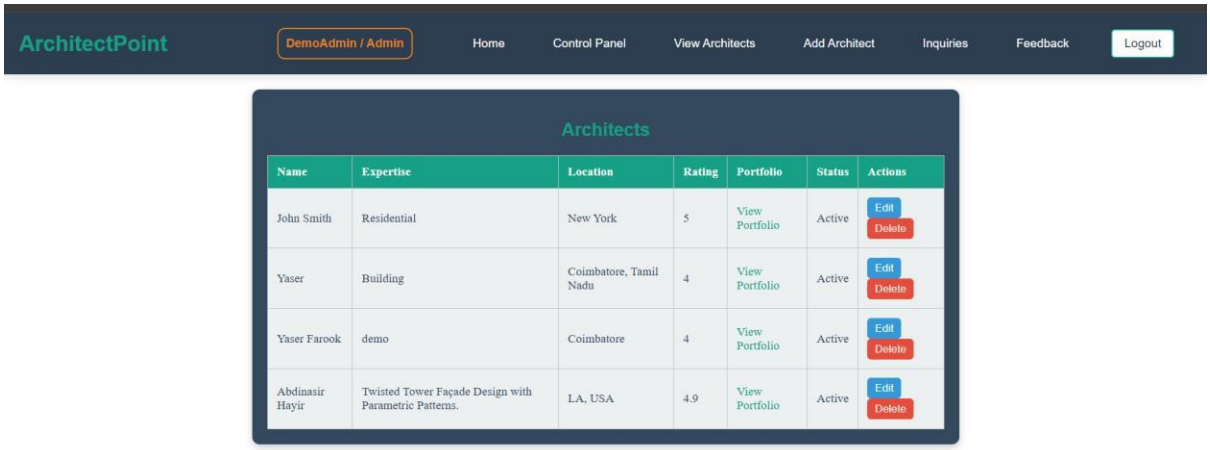
On the navbar, a menu "View Architects".



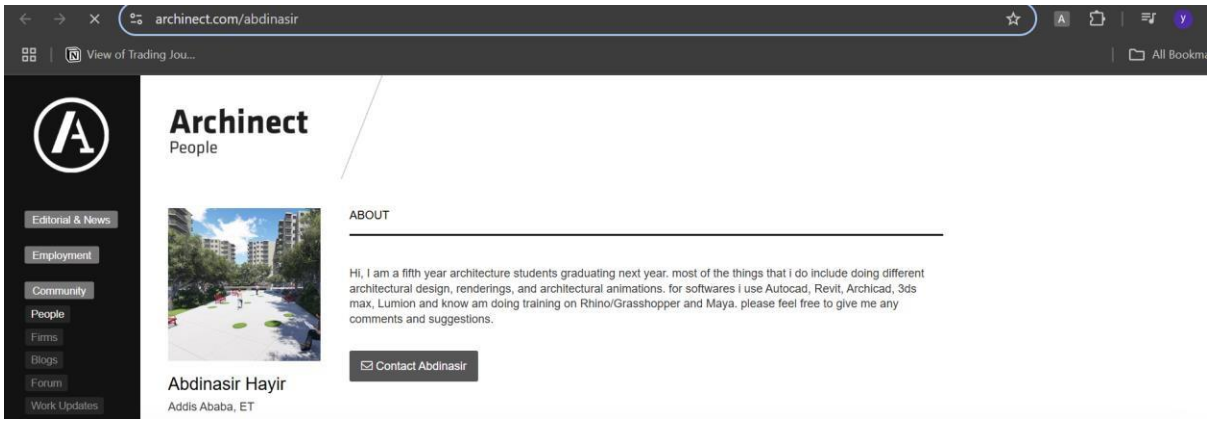
Clicking on "View Architects" will navigate to the **admin-view-architects component**, which displays all architects details in a table format with the heading "Manage architects". Additionally, a search feature is provided to search based on Architect Name and filter based on categories available.

If no data is available, then “No Architects available. Add some Architect to view here." should be displayed.

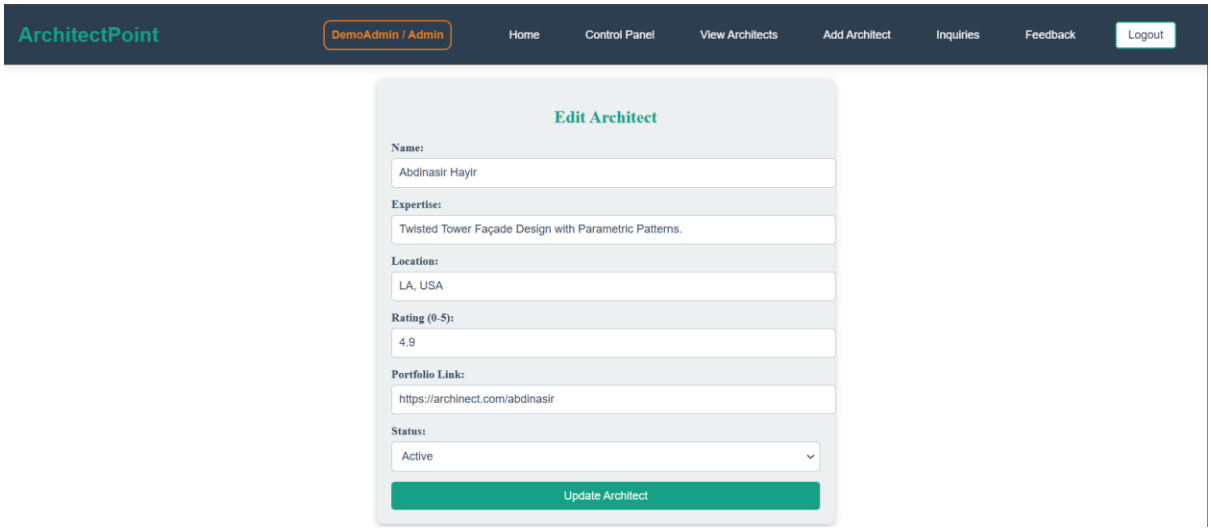




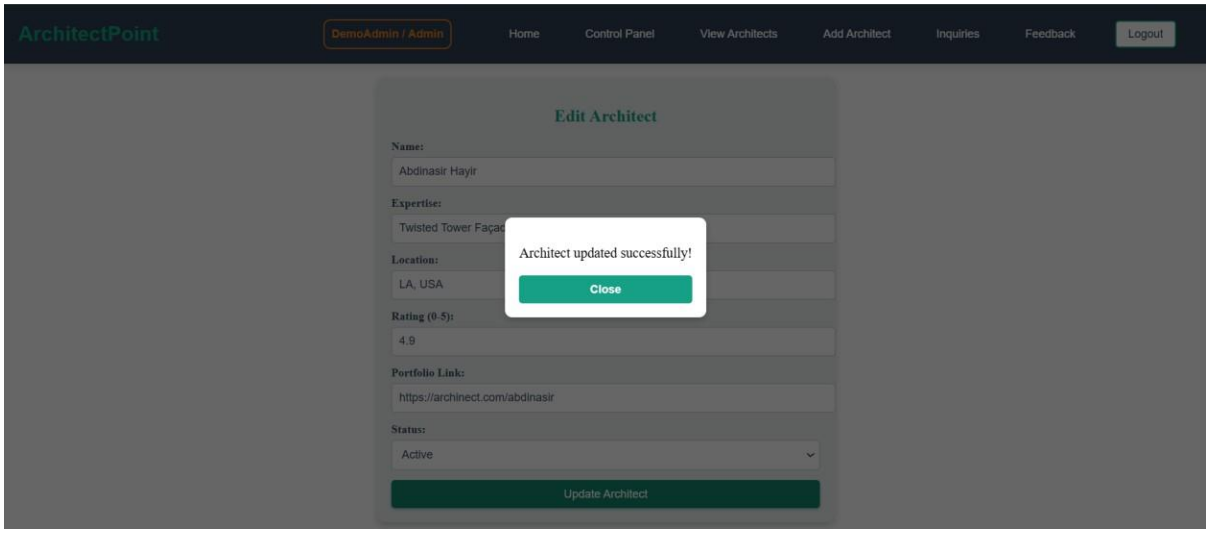
Clicking on View portfolio it will redirect to that website.



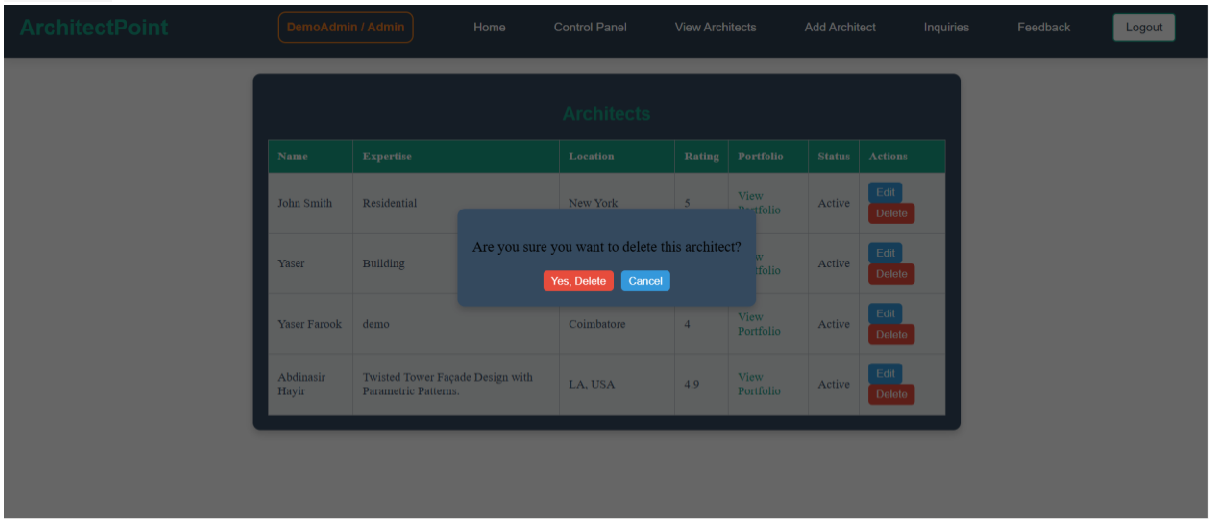
Clicking the edit button will navigate to the **admin-edit-architect component**, which displays an editing form with pre-populated data of the selected architect. Validations are performed for all the form fields.



Upon clicking the "Update Architect" button, if the operation is successful, a popup saying "Architect Updated Successfully!" will be displayed. Clicking "OK" will redirect to the **admin-view-architect** component.



On clicking the "Delete" button, a pop-up should be displayed with confirmatory message to delete the data.



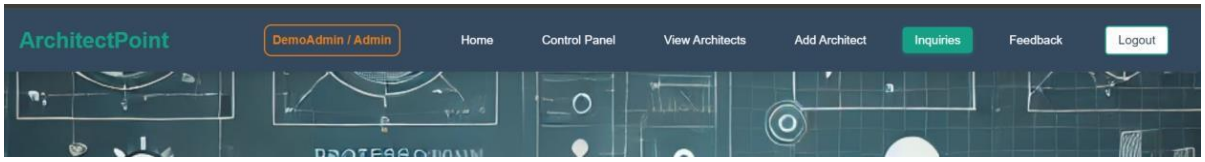
Clicking “Yes, Delete” button will delete the record. Clicking “Cancel” button will close the popup.



After successful deletion, the table should be refreshed as mentioned in the above images.

Admin View Inquiry

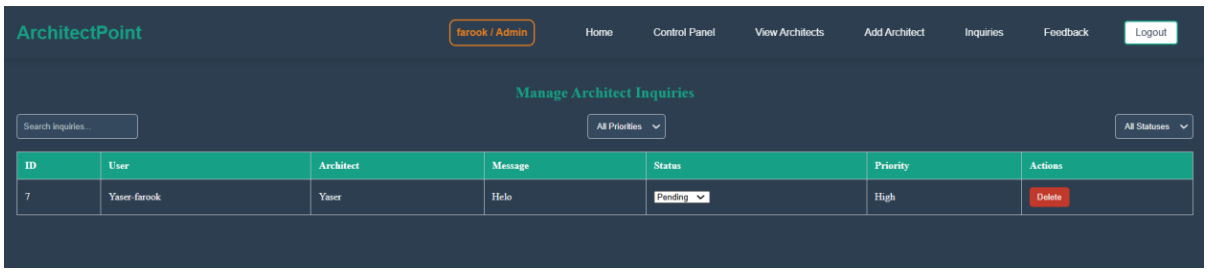
Clicking on “Inquiries" from the navbar will navigate to the **admin-view-inquiry component**, which displays all Inquiries by all users.



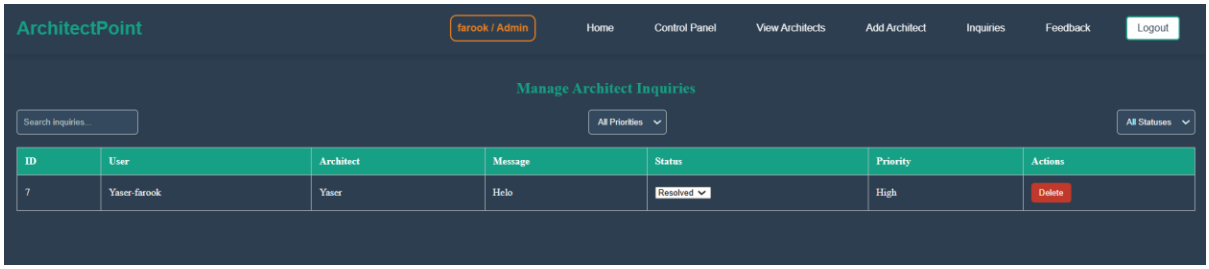
If no data is available, then "No inquires found" should be displayed.

No inquiries found.

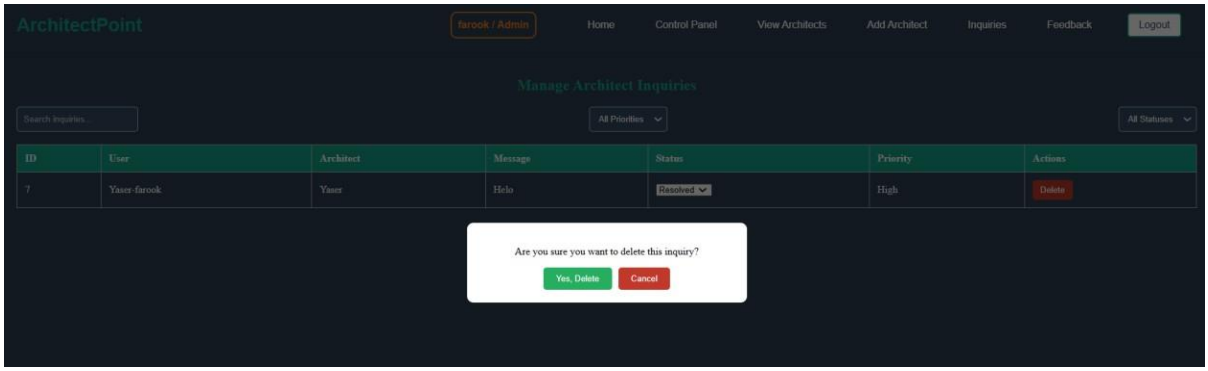
If data is available, the inquiry by all users will be displayed in table format. Additionally, features such as the admin being able to search based on Architect Name and filter the data based on Priority and statuses.



The admin can delete the users inquiry



Admin can also change the status of the inquiry but clicking this dropdown button.

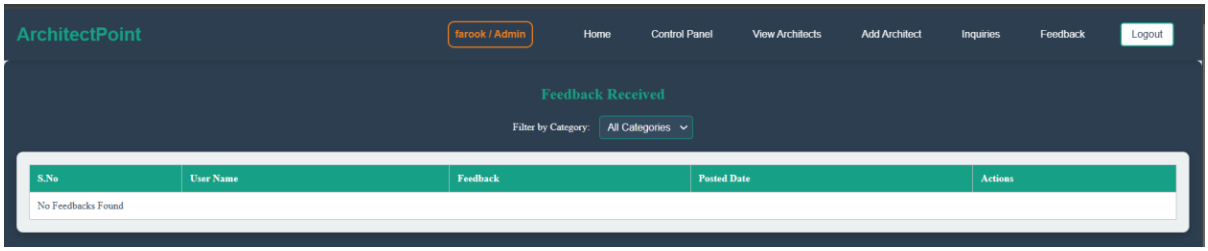


Admin can also delete the inquiry by clicking the delete button and a pop up shows up. Once “Yes, Delete” is click the inquiry will get deleted .

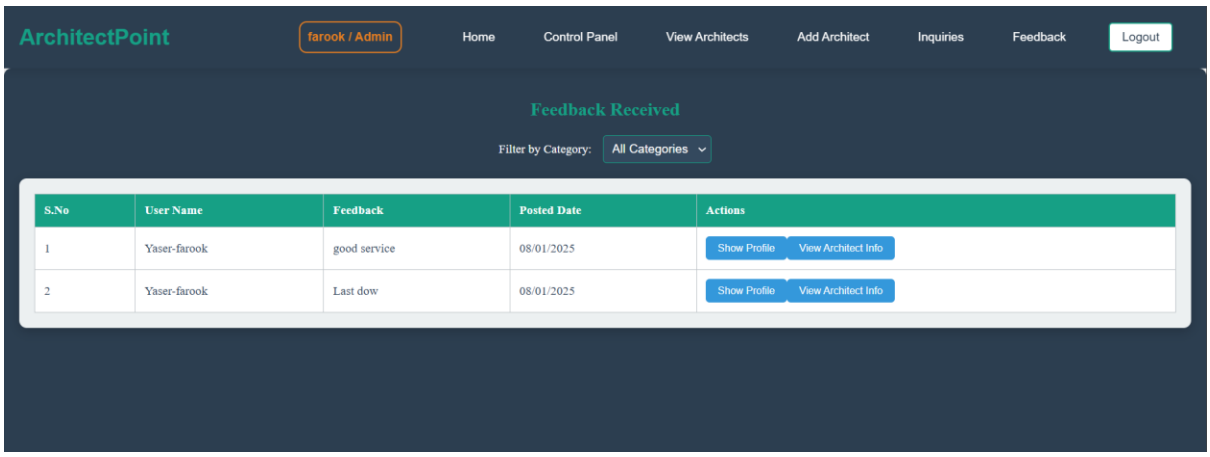
Admin View Feedbacks:

Clicking on "Feedbacks" from the navbar will navigate to the **admin-view-feedback component**, which displays all feedbacks posted by all users.

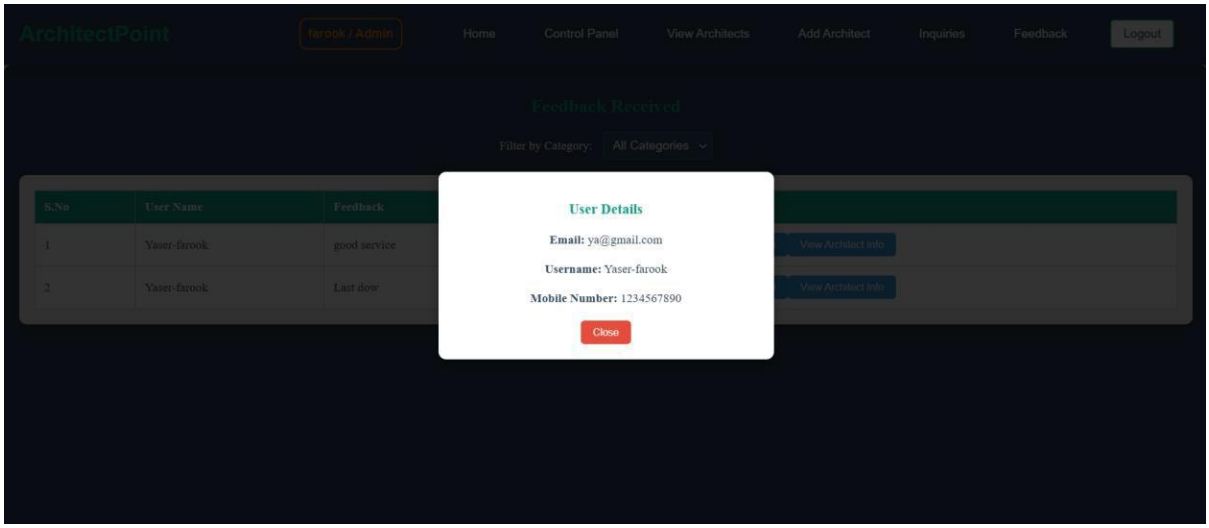
If no data is available, then "No data found" should be displayed.



If data is available, the feedbacks posted by all users will be displayed in table format. Admin can filter based on category.

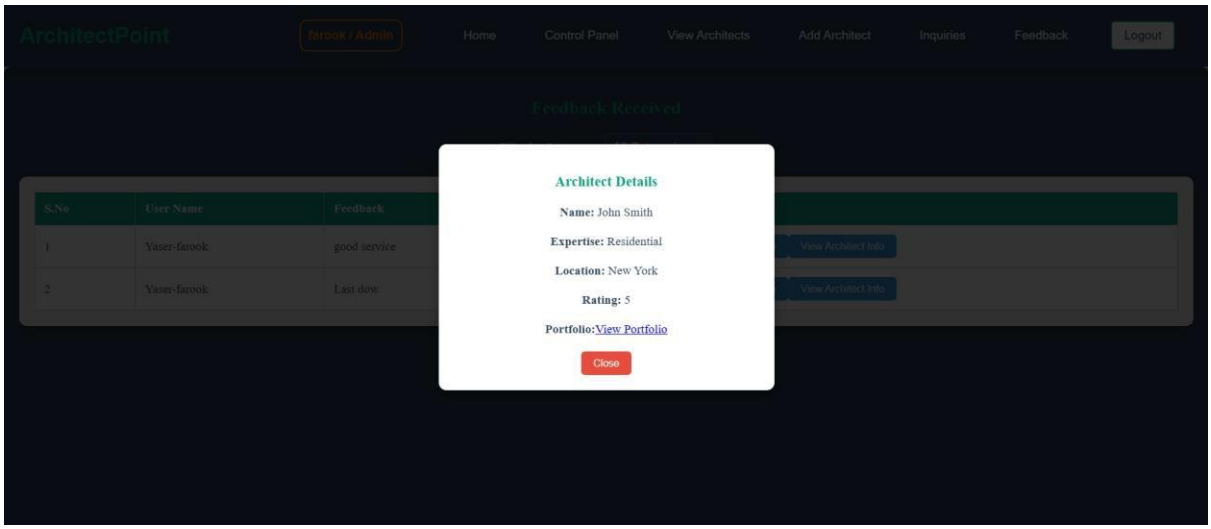


Clicking on "Show Profile" will display additional details about the user in pop-up modal.



On clicking the "Close" button, will close the pop-up modal displayed.

Clicking on "View Architect Info" will display additional details about the user in pop-up modal.



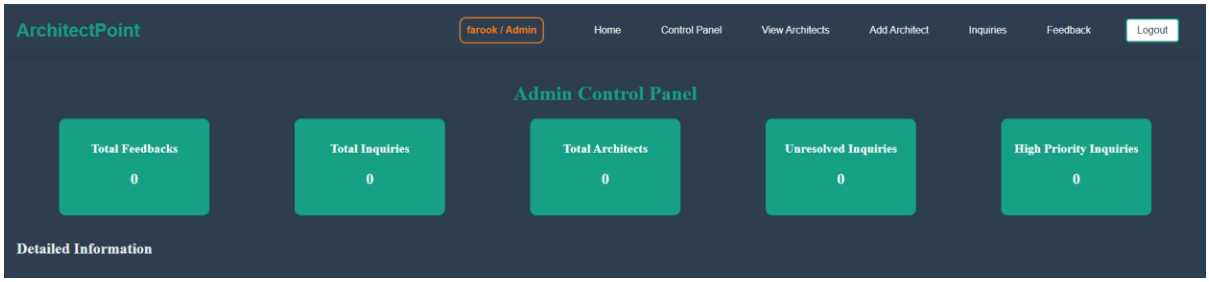
On clicking the "Close" button, will close the pop-up modal displayed.
On clicking the "View Portfolio" button it should redirect to architect's portfolio.

Admin Control Panel:

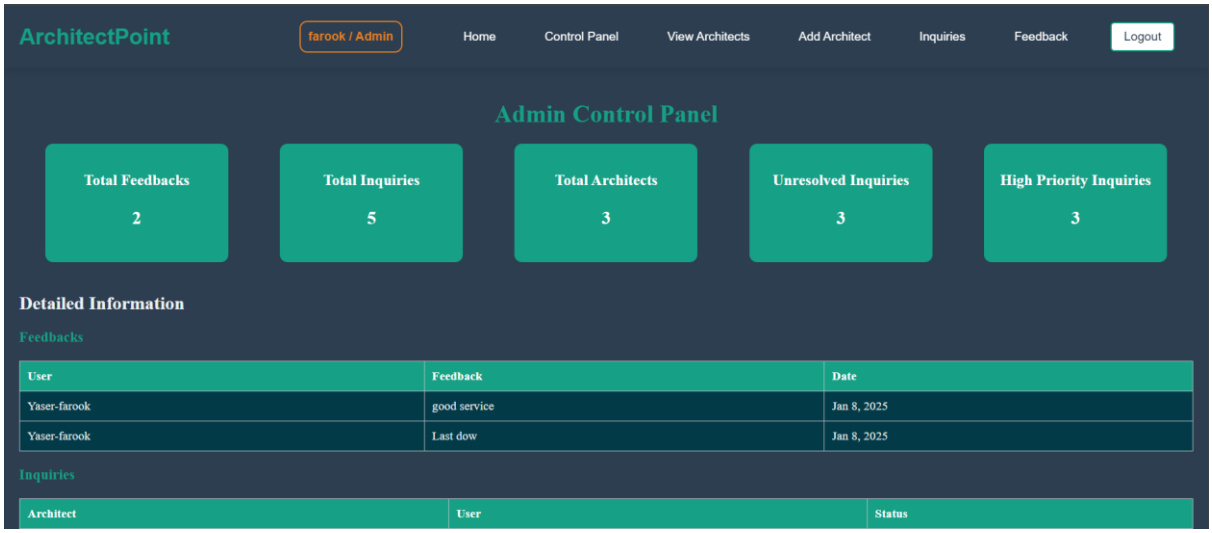
Clicking on "Control Panel" from the navbar will navigate to the **admin-control-panel component**,



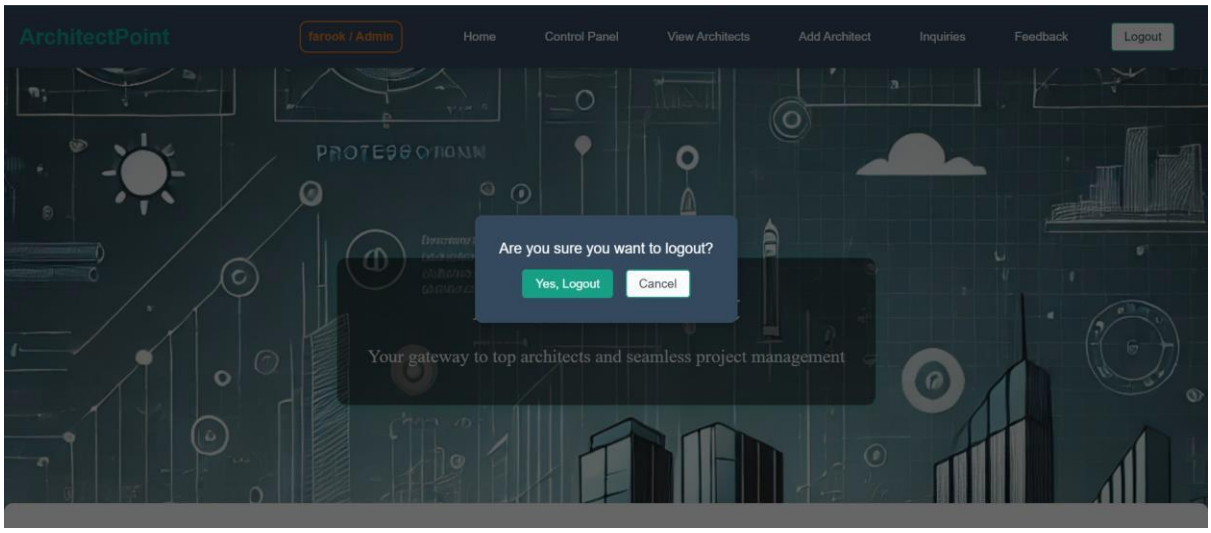
If no data is available to display , then "No data available" will be displayed.



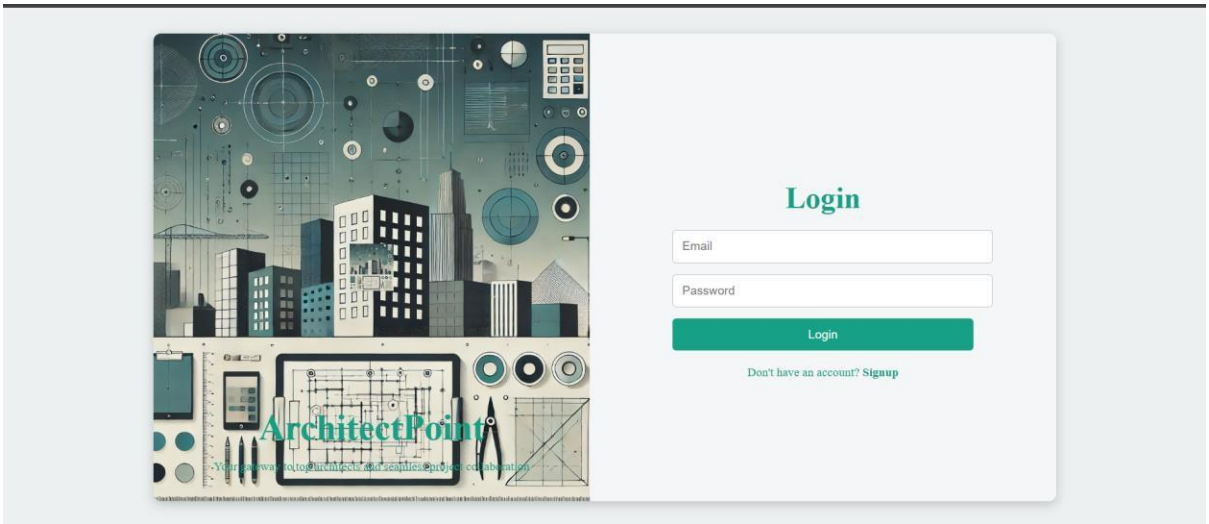
If Data is available displays users total inquiries, high-priority Inquires and un resolved inquiries and total feedbacks.



On clicking the "Logout" button, a pop-up should be displayed with confirmatory message to logout the user.

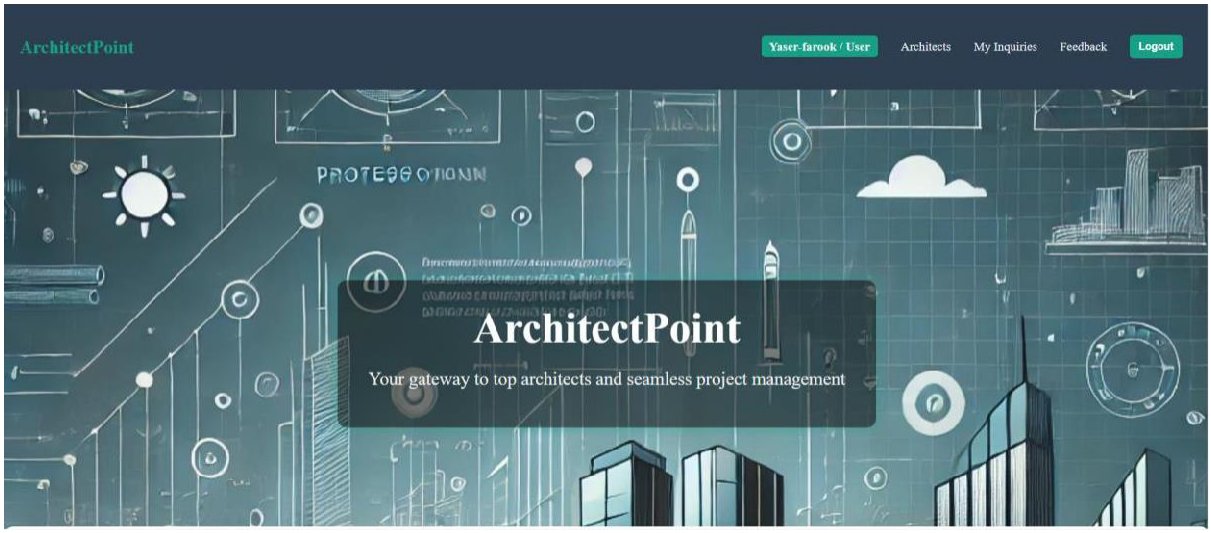


Clicking "Yes, Logout" will navigate to the login component.

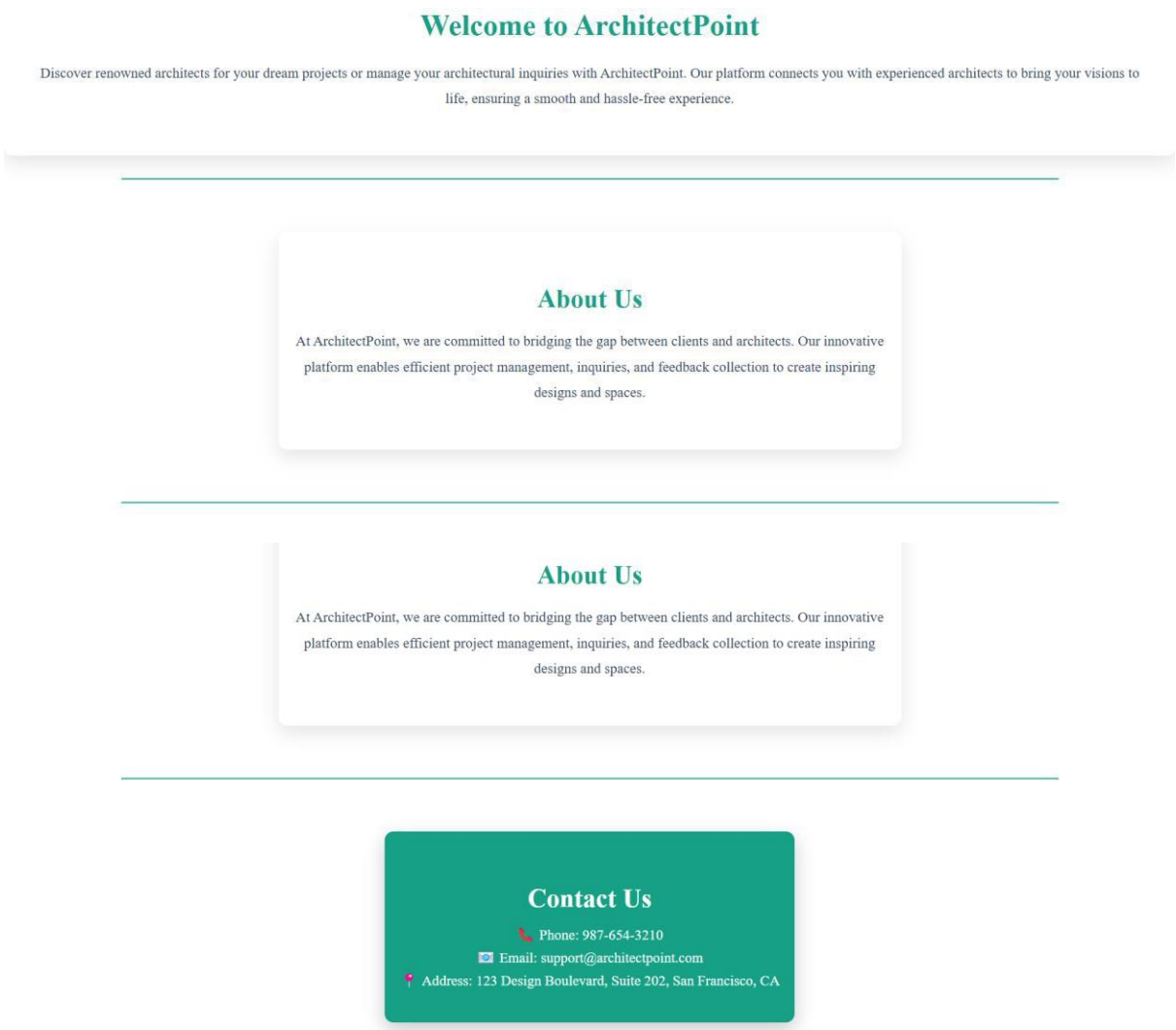


User side:

Upon successful login, if the user is an admin, the **(adminnav component)** will be displayed. If the user is a regular user, the **(usernav component)** will be displayed. Additionally, the role-based navigation bar will also display login information such as the username and role.

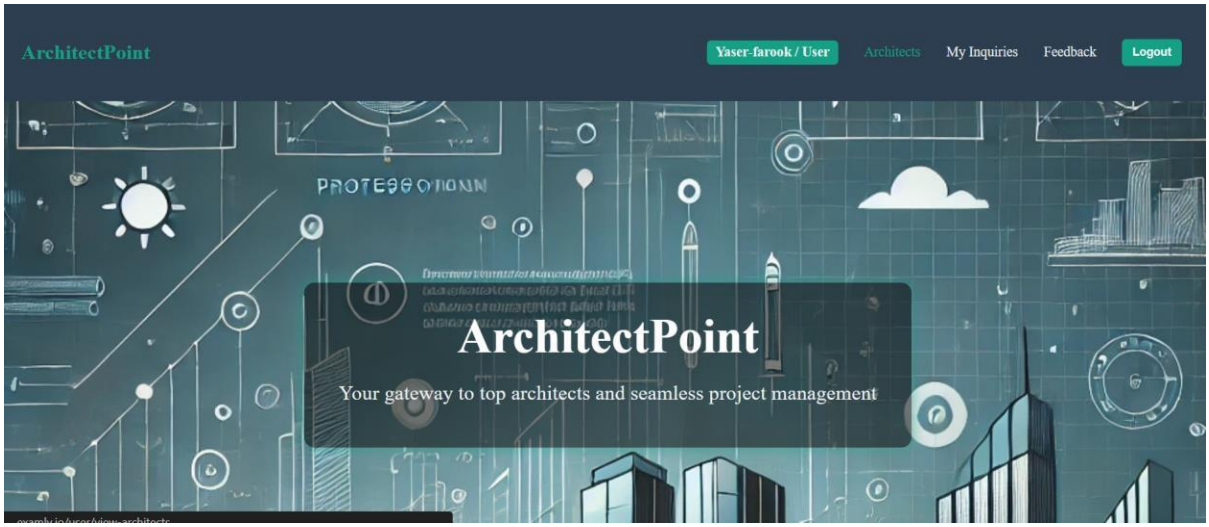


homePage Component: This page is used to display the information about ArchitectPoint application. On clicking the '**Home**' tab, user can view the information about the application.(home.webp provided in images folder under assets)

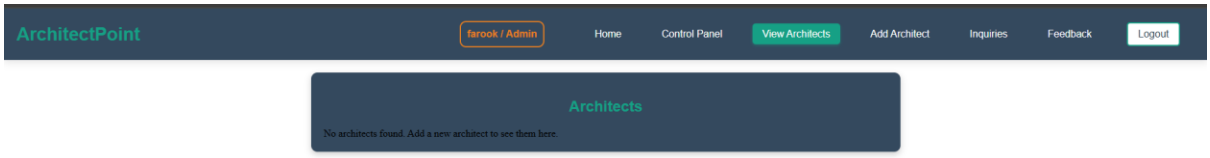


User View Architects:

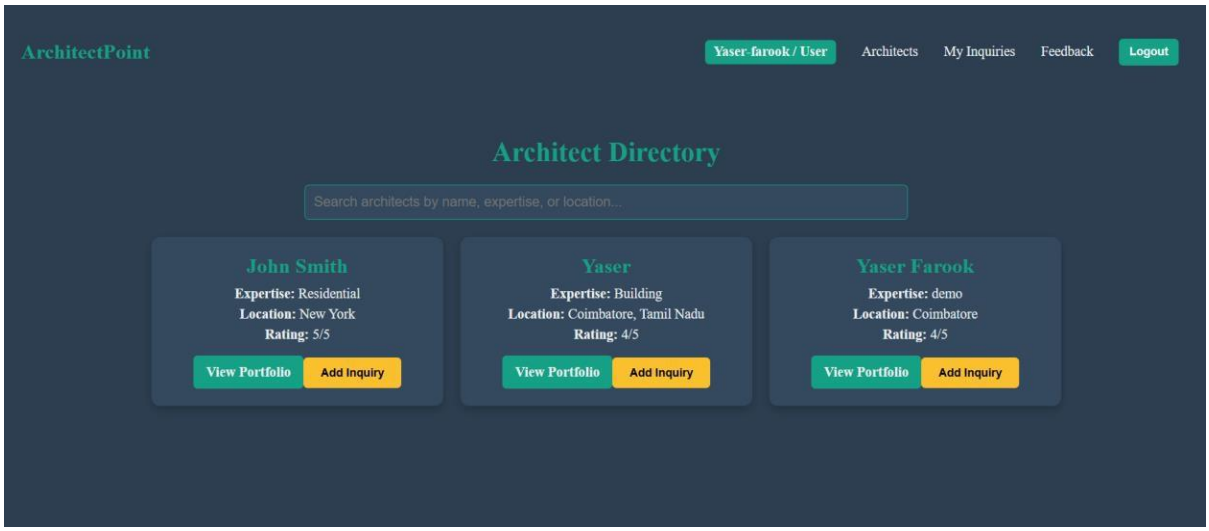
On clicking on "Architects" item in the navbar, Can view the architects.



Clicking on "Architects" from the navbar will navigate to the **user-view-architects** component, which displays all architects posted by admin.

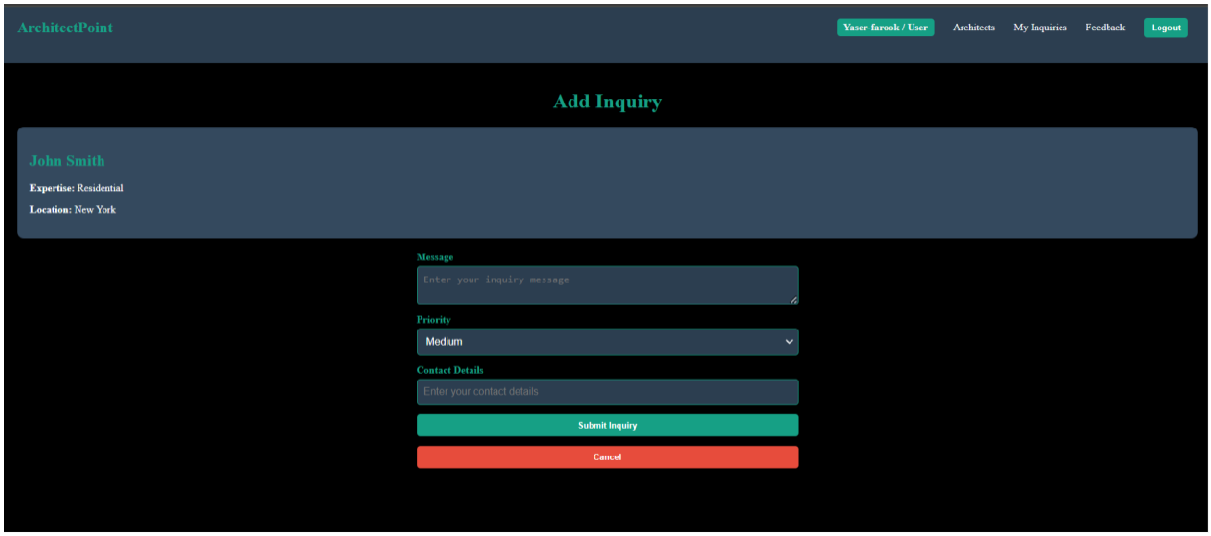


If no data is available, then "**No architects found. Add new architect to see them**" should be displayed.

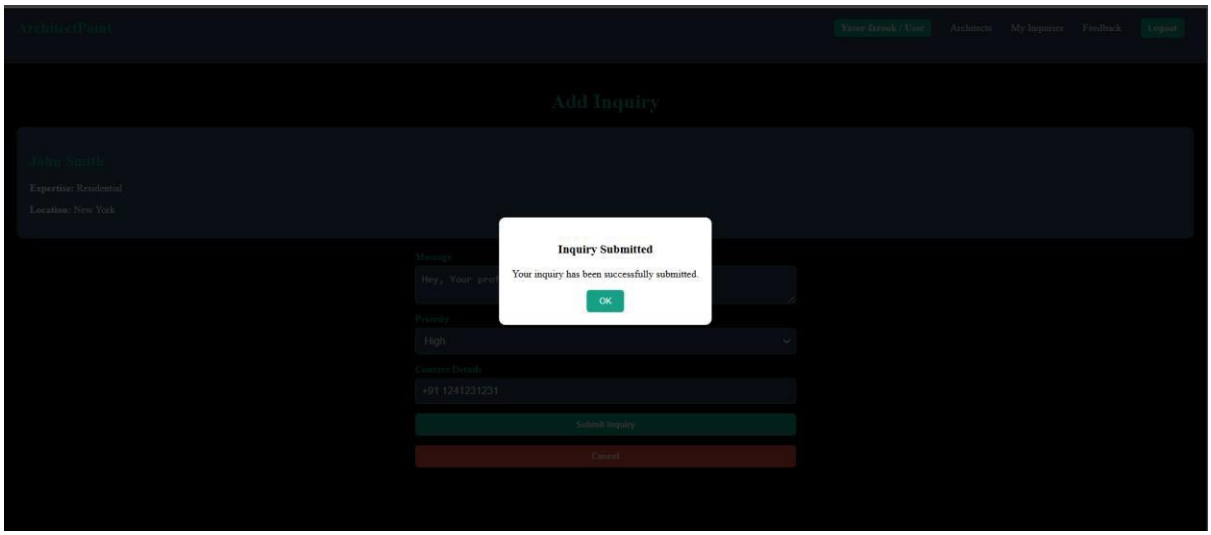


There will be a two-button named “Add Inquiry” to inquire about the architect, and “View-Portfolio” to view the website of architect.

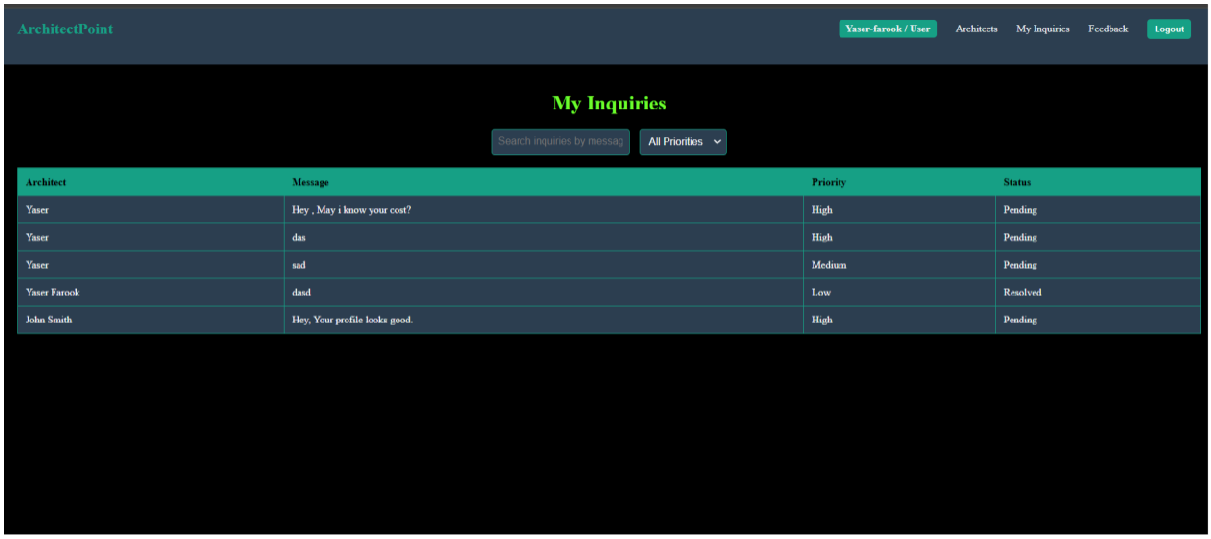
Clicking the "Add Inquiry" button will navigate to the **user-add-inquiry component,** which displays a form with the heading "**Add Inquiry**”.



Clicking on “**Submit Inquiry**” button will allow user to submit their inquiry. if the operation is successful, a popup message saying "Your Inquiry has been Successfully Submitted." should be displayed.



Clicking "Ok" will redirect to the user-view-inquiries component.



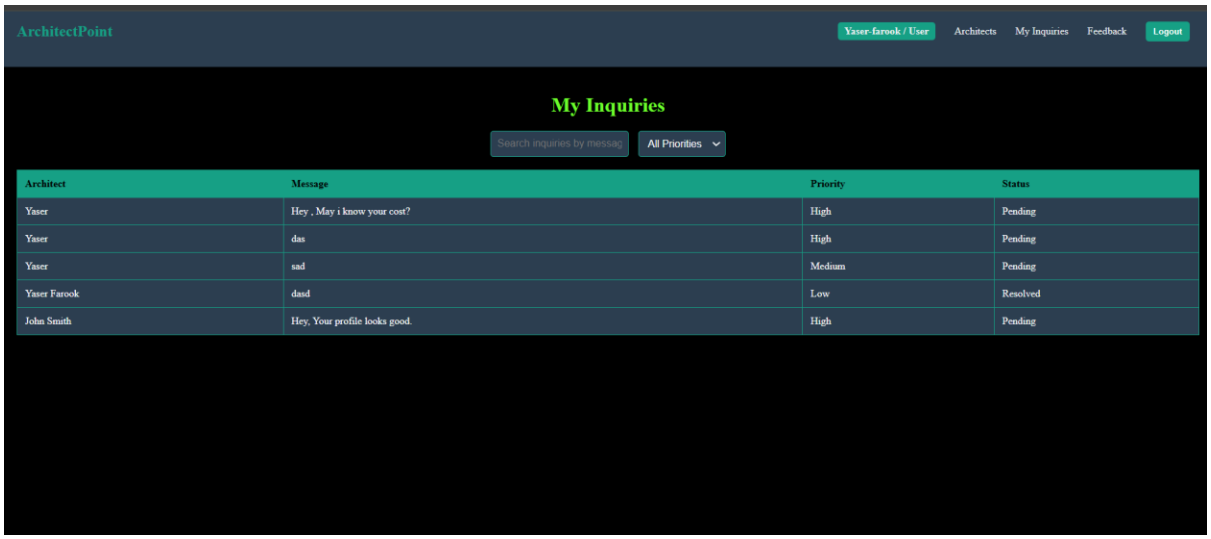
User Applied Inquiry:

On the navbar, a menu named with "My Inquiries".



Clicking on "My Inquiries" from the navbar will navigate to the **user-view-inquiries component**, which displays the user's applied Inquiries.

If data is available, the Inquiries will be displayed in table format.



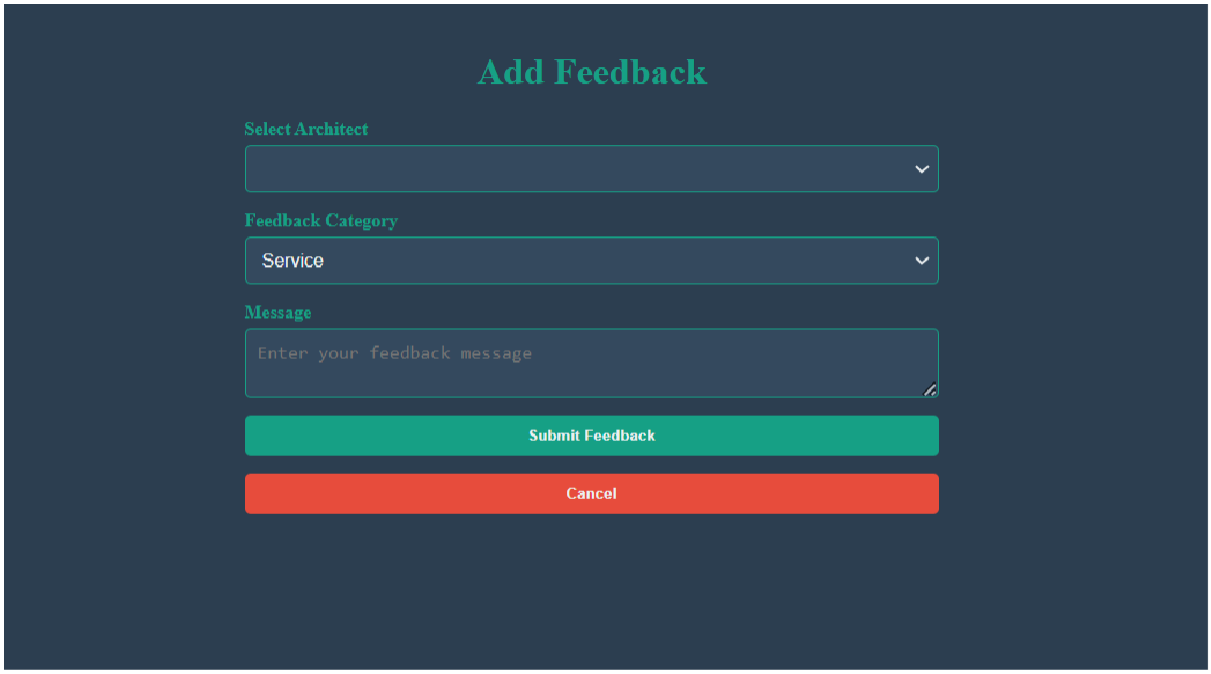
There will be another column named “Status”, Once the admin changes the status, the user inquiry will be updated.



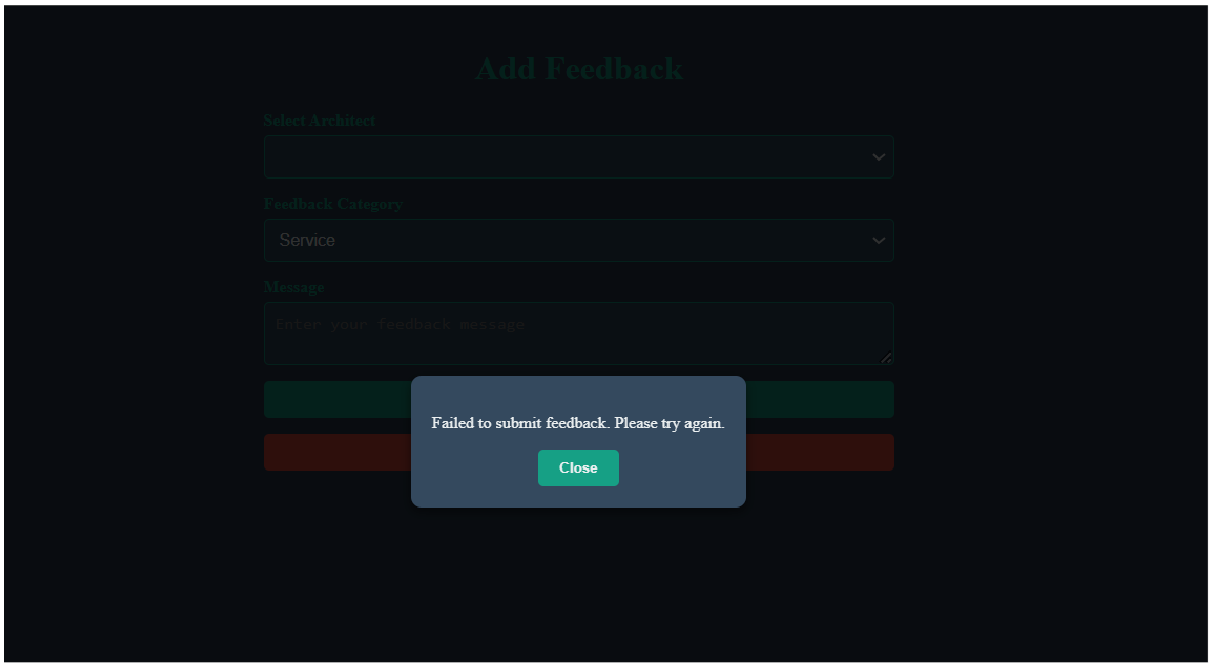
Once the admin gave the response, the user’s inquiry will get updated and the status will moved to resolved stage.

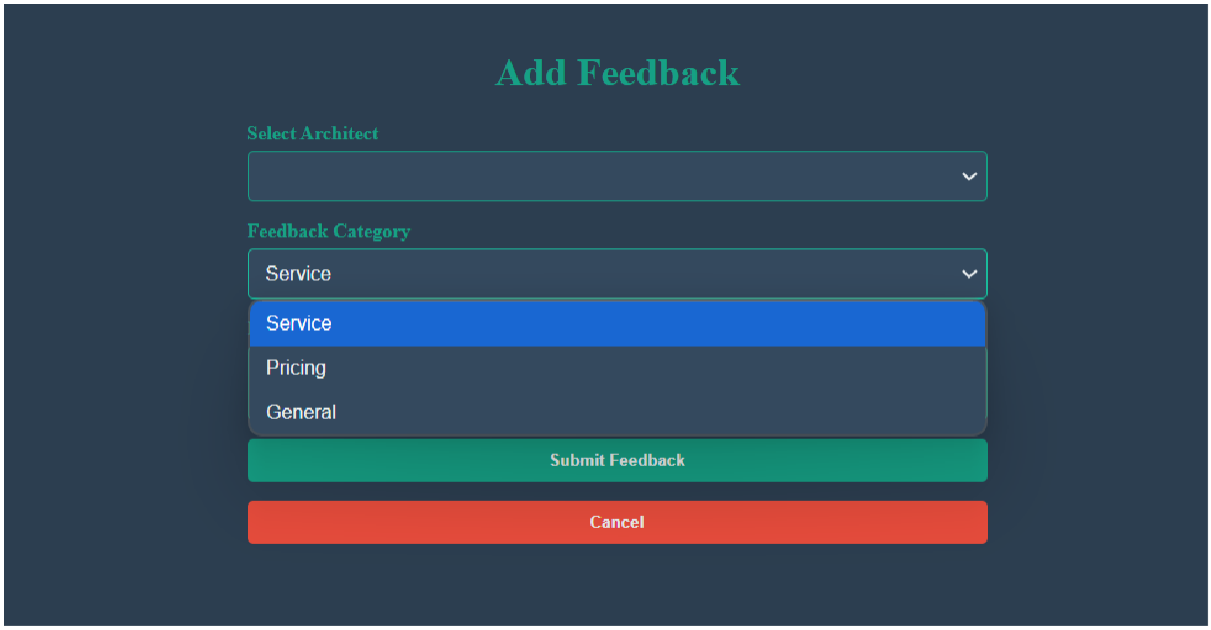
On navbar there will be another menu named “Feedback”. On clicking on Feedback, will navigate to the **user-add-feedback component**, which displays the form to post feedback with heading as "Submit Feedback"



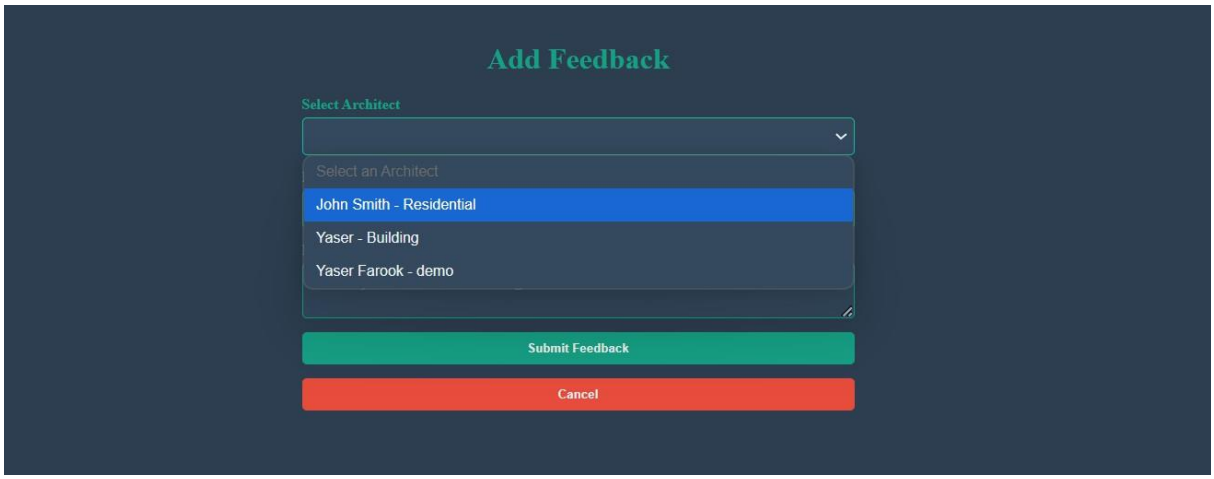


Clicking the "Submit" button with empty field will display a Pop up message.



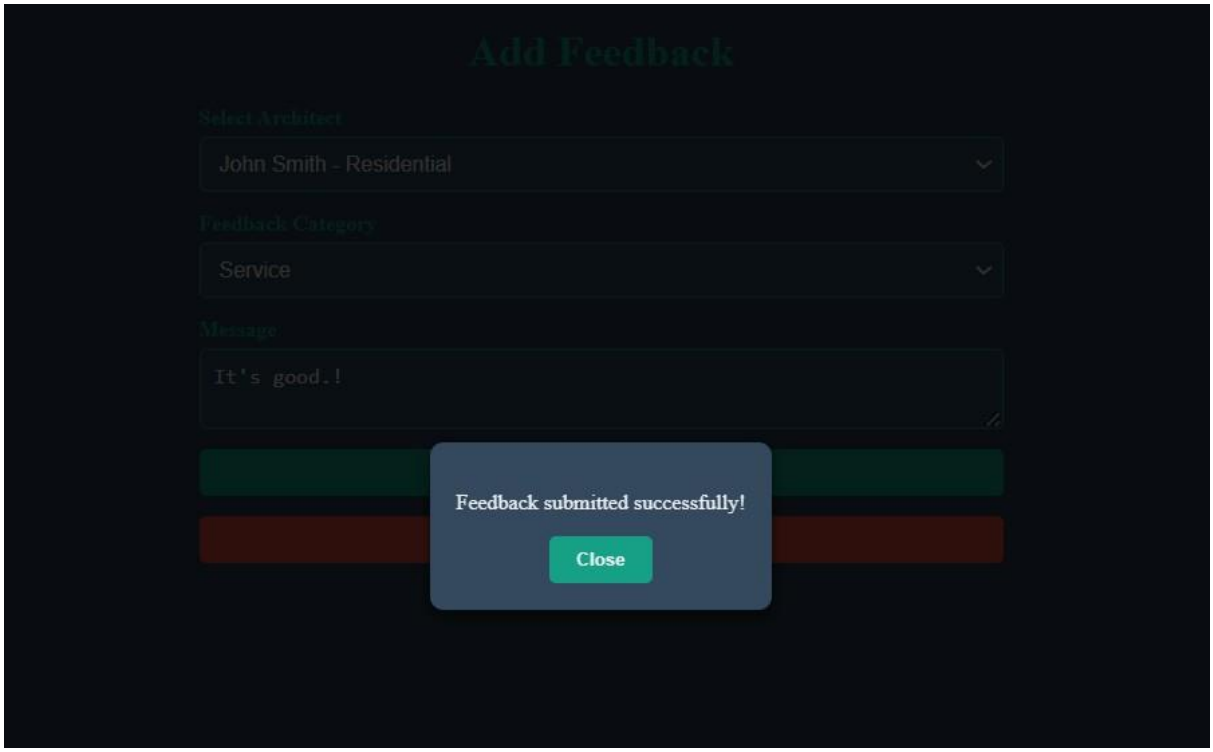


Once message got filled , you can select the category but clicking the drop-drop button.

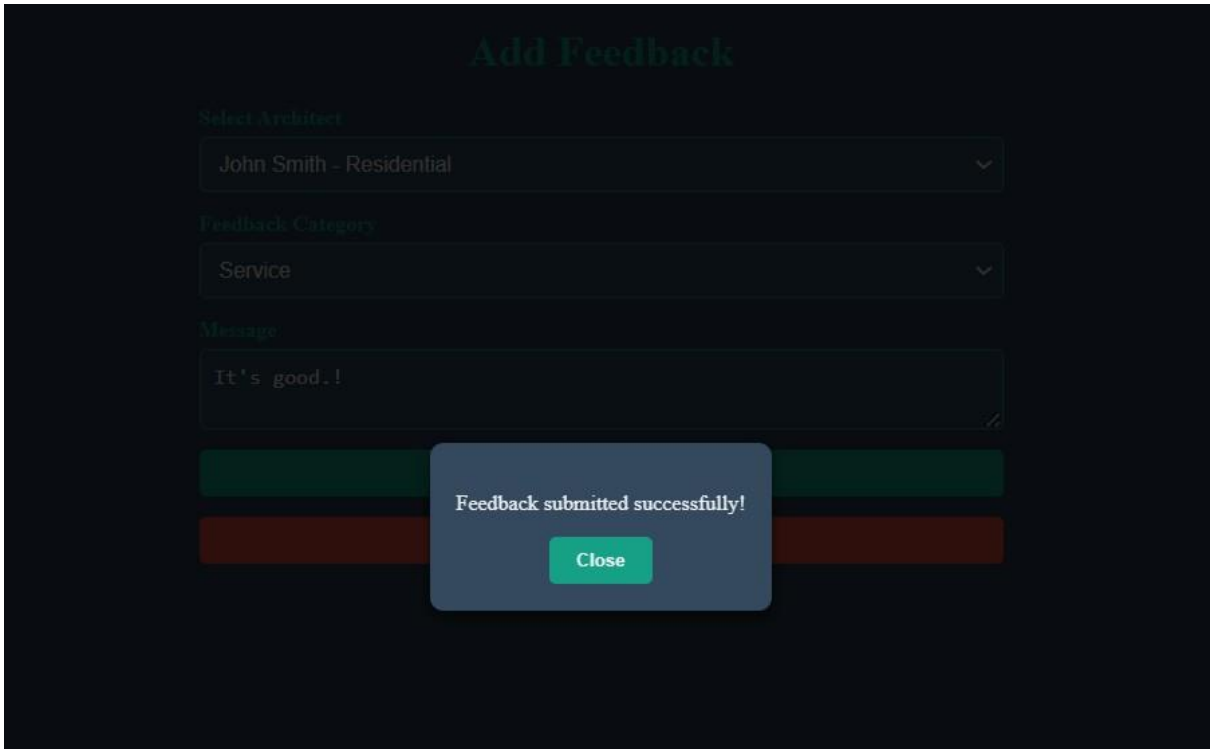


Once the message and category got filled, you can select the architect, by clicking the drop-down.

Upon clicking the "Submit Feedback" button, if the operation is successful, a popup message saying "Feedback submitted Successfully!" should be displayed.

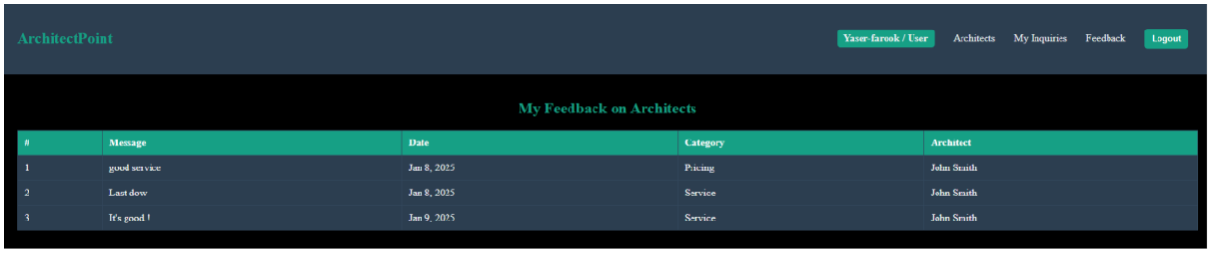


User View Feedback:

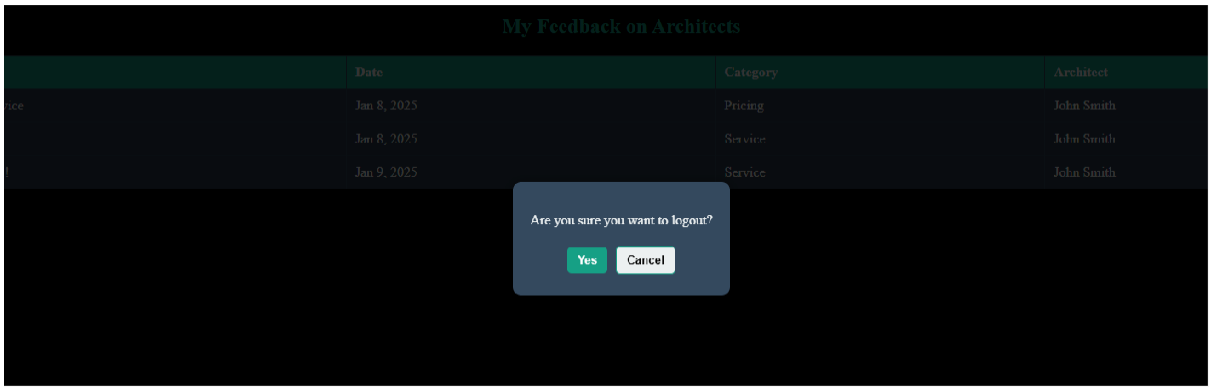


Once you click ok , the page will navigate to

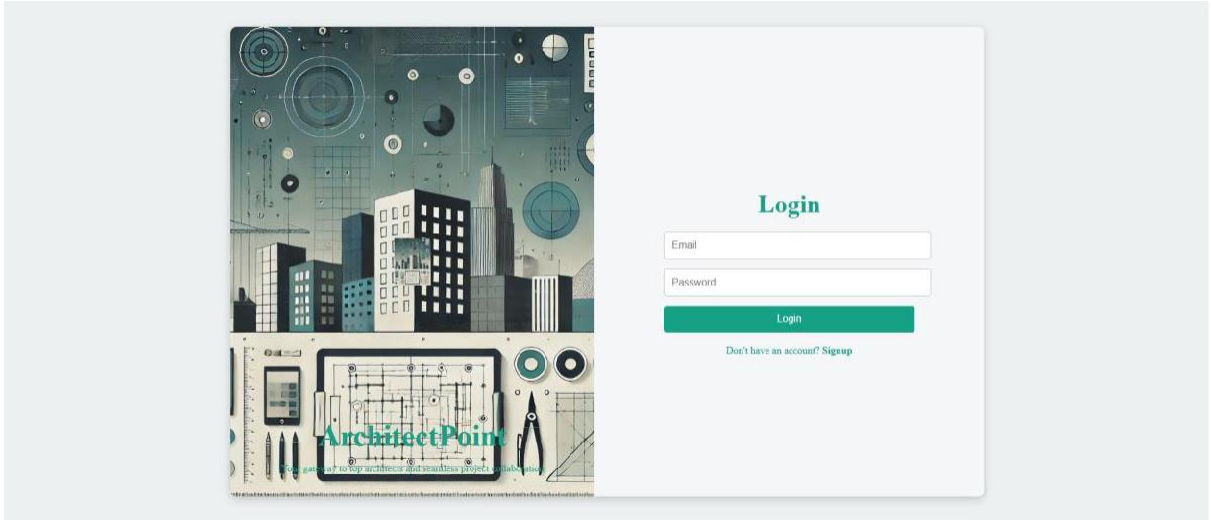
Once the feedback is posted it will navigate to the **user-view-feedback component**, which displays posted feedback with heading as "My Feedback"



On clicking the "Logout" button, a pop-up should be displayed with confirmatory message to logout the user.



Clicking "Yes, Logout" will navigate to the login component.



error component

This page displays an error message stating "Something Went Wrong".



Platform Prerequisites (Do's and Don'ts):

1. The angular app should run in port 8081.
2. The springapp should run in port 8080.
3. To incorporate Spring Security into the application, utilize either basic authentication or JWT authentication within the project workspace. For auto-evaluation purpose, before submitting the code, please verify that all security-related files, including the security dependency in the pom.xml, have been commented out. Alternatively, include GET: "/api/architects" and GET: "/api/feedback" APIs in permitAll() in the security config to allow all users to access the APIs

Key points to remember:

1. The id (for frontend) and attributes(backend) mentioned in the SRS should not be modified at any cost. Failing to do may fail test cases.
2. Remember to check the screenshots provided with the SRS.
3. Strictly adhere to the proper project scaffolding (Folder structure), coding conventions, method definitions and return types.
4. Adhere strictly to the endpoints mentioned in API endpoints section.
5. Don't delete any files in a project environment.

HOW TO RUN THE PROJECT:

BACKEND:

Open the terminal and follow the commands below.

- **cd springapp**

To run the application in port 8080

- **mvn spring-boot: run**

If the same error persists clean the project and build again

To Work with MysqlServer:

(Open a New Terminal) type the below commands

mysql -u root –protocol=tcp -p

password: **examly**

Note:

1. Please ensure that the application is running on port 8080 before clicking the "Run Test Case" button.
2. Database Name should be **appdb**

FRONTEND:

Open the terminal and follow the commands below.

Step 1: Switch to Node.js version 20 using the command: nvm use 20

- **Use "cd angularapp**" command to go inside the angularapp folder
- Install Node Modules - "**npm install**"

Step 2:

- Write the code inside src/app folder
- Create the necessary components
- To create Service: "**npx ng g s <service name>**"
- To create Component: "**npx ng g c <component name>**"

Step 3:

- Click the **Run Test Case** button to run the test cases

Note:

- Click PORT **8081** to view the result / output.
- If any error persists while running the app, delete the node modules and reinstall them.

AI Integrated Feature (Phase 2)

As part of the LTM AI-integration initiative, ArchitectPoint has been enhanced with an AI-driven feature powered by the Google Gemini API. This feature reuses a shared GeminiService (text generation) and degrades gracefully to a non-AI fallback when the API key is not configured, so existing functionality and test cases remain intact.

AI Feature Overview:

Per-Architect AI Review Summary (LLM Summarization): For any architect, all feedback submitted about them is aggregated by Gemini into a concise digest covering strengths, areas to improve, and an overall AI quality score. This gives an Admin an at-a-glance read of an architect's overall reputation without having to read through every feedback entry manually. The summary is generated on demand each time it's requested, so it always reflects the current set of feedback for that architect.

AI Backend Endpoints:

Architect Review Summary

URL: /api/ai/architect/{architectId}/feedback-summary
Method: GET
Role: Admin
Response: Returns response code 200 with an AI summary object containing architectId, architectName, feedbackCount, averageRating, summary, strengths, improvements, and aiScore. If the architect has no feedback yet, returns the object with feedbackCount 0, a message stating no feedback has been submitted, and aiScore null.

AI Service Layer:

GeminiService (service folder):

- embed(text): Returns a semantic float[] embedding via the Gemini embedding model. Not used by this feature; kept on the service for parity/reuse if another AI feature is added later.
- generate(prompt): Sends the given prompt to Gemini's generateContent endpoint and

returns the generated text. Used by AiService to obtain the summary, strengths, improvements, and score. Returns null if the call fails or Gemini is not enabled, so callers can fall back gracefully.
- isEnabled(): Returns true when a non-empty gemini.api.key is configured; returns false otherwise, in which case generate() short-circuits to null without attempting a network call.

AiService (service folder):

- summarizeArchitectFeedback(architectId): The single entry point for this feature. Looks up the architect and all feedback linked to them, then builds the summary. If no feedback exists, returns immediately with an informational message and no AI score, without calling Gemini. The average rating shown in the summary is read directly from the Architect entity's own rating field. If Gemini is disabled, returns a plain message noting that feedback exists but an API key is needed to generate a summary. Otherwise, builds a prompt from all of the architect's feedback text (each entry prefixed with its category — Service, Pricing, or Quality) instructing Gemini to produce a 2-3 sentence overview, a list of strengths, a list of areas to improve, and an overall quality score from 0-100, then parses the JSON response. If the response is missing, malformed, or fails to parse, falls back to a plain feedback-count message rather than propagating an error, so a Gemini outage never breaks the admin view.

Frontend (Angular) Additions:

services/ai.service.ts

- Dedicated Angular service for ArchitectPoint's AI endpoint.
- getArchitectFeedbackSummary(architectId) - calls GET /api/ai/architect/{architectId}/feedback-summary to fetch the AI-generated summary for one architect.
- Sends the Bearer token from localStorage (via AuthService) in the request headers, matching the authentication pattern used by the app's other services.

models/ai.model.ts

- Defines the ArchitectFeedbackSummary interface returned by the AI endpoint: architectId (number), architectName (string), feedbackCount (number), averageRating (number or null), summary (string), strengths (string array), improvements (string array), and aiScore (number or null, null when no feedback exists yet).

admin-view-architect component

- Adds an "✨ AI Summary" button in the Actions column of the architects table, alongside the existing Edit and Delete buttons.
- Clicking it opens a modal showing the architect's name, a loading state while the summary is generated, and once loaded: the feedback count, average rating, AI score, the summary paragraph, and bulleted lists of strengths and areas to improve.
- Displays the backend's informational message gracefully for the no-feedback and Gemini-disabled cases, rather than showing an empty or broken panel.

Configuration (application.properties):

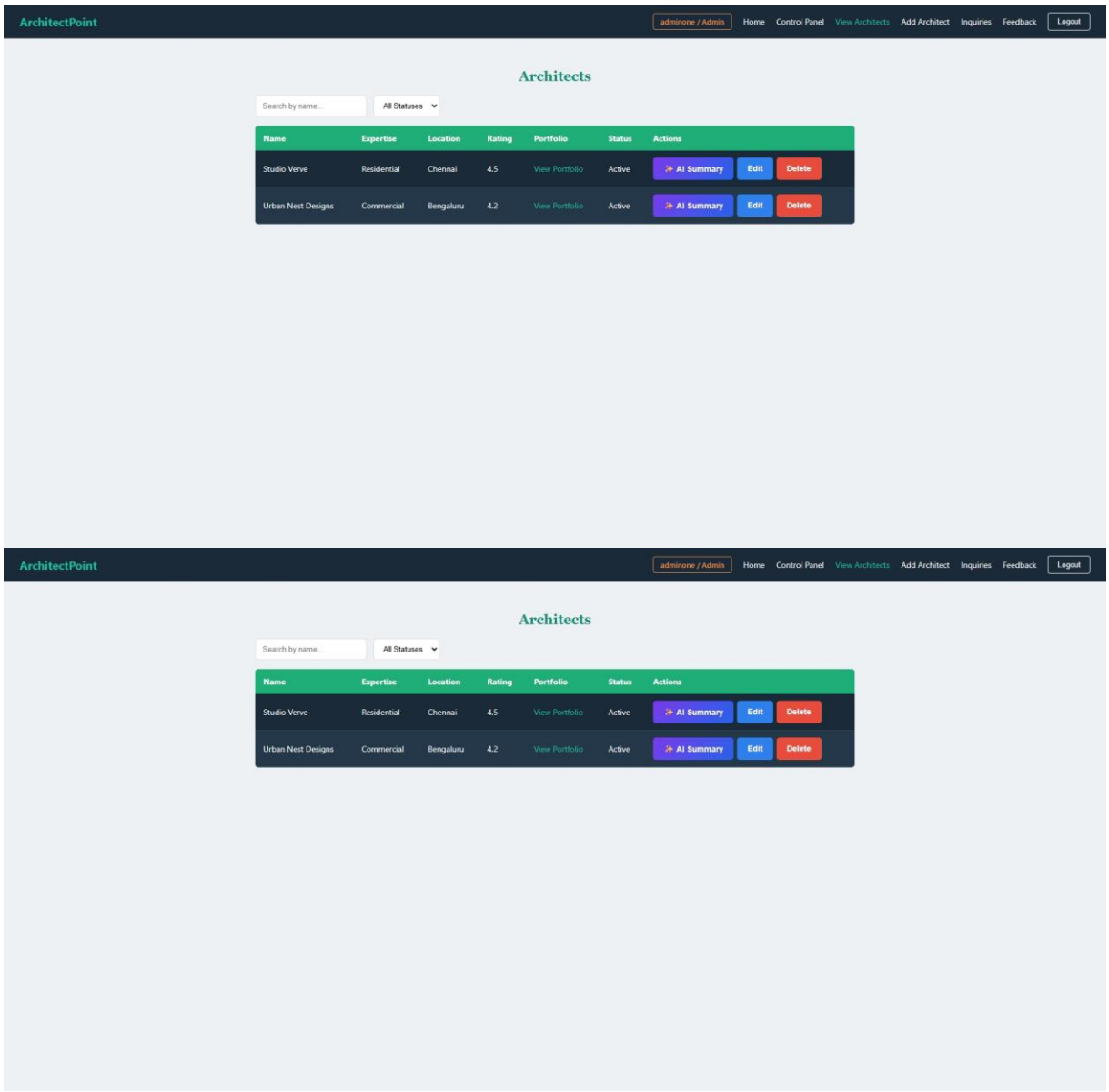
Property	Description
gemini.api.key	Google Gemini API key. If empty, the summary falls back to a plain informational message with no AI score.
gemini.embedding.model	Embedding model id, e.g. text-embedding-004.
gemini.generation.model	Generation model id, e.g. gemini-2.5-flash.

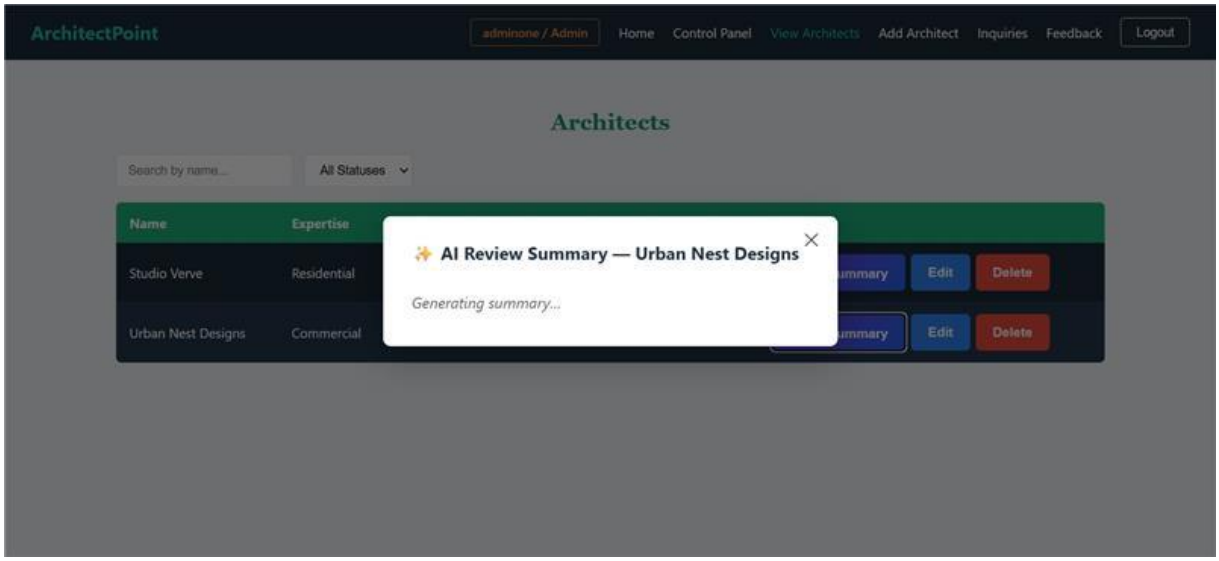
Platform Note (Security):

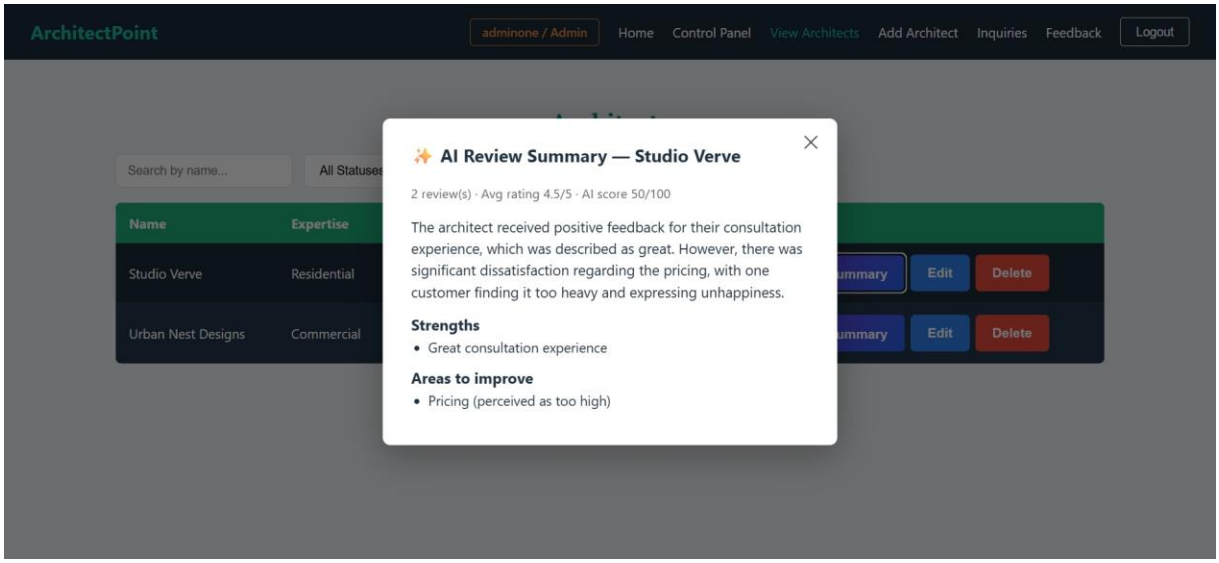
The AI endpoint is grouped under /api/ai/architect/** and restricted to Admin, consistent with the existing GET /api/architects/** rules governing architect management. The feature is purely read-only and never writes to any entity.

Sample AI Integration Screenshots:

Admin can view the AI-generated Review Summary for an architect directly from the Architects list, without needing to read through every individual feedback entry.




ArchitectPoint
adminone / Admin
Home
Control Panel
View Architects
Add Architect
Inquiries
Feedback
Logout
Architects
Search by name...
All Statuses
Name
Expertise
Studio Verve
Residential
Urban Nest Designs
Commercial
✨ AI Review Summary — Urban Nest Designs
Generating summary...
Edit
Delete


ArchitectPoint
adminone / Admin
Home
Control Panel
View Architects
Add Architect
Inquiries
Feedback
Logout
Search by name...
Name
Expertise
Studio Verve
Residential
Urban Nest Designs
Commercial
✨ AI Review Summary — Studio Verve
2 review(s) · Avg rating 4.5/5 · AI score 50/100
The architect received positive feedback for their consultation experience, which was described as great. However, there was significant dissatisfaction regarding the pricing, with one customer finding it too heavy and expressing unhappiness.
Strengths
• Great consultation experience
Areas to improve
• Pricing (perceived as too high)
Edit
Delete